

PDF

Kaustuk_CV
(3)

i wanted you to prepare me a fully detailed notbook where you include every thing discussed in gaurav sen system design playlist in details , with why , what how and other in details in simple way.and atleast 15 full solution of



The Complete Engineering Notebook: Distributed Systems, System Design (HLD & LLD), and Modern Enterprise Backend Architecture

1. Core Distributed Systems Foundations (The Gaurav Sen Curriculum)

1.1 Vertical vs Horizontal Scaling

Vertical Scaling (Scale-Up): Adding more compute resources (CPU cores, RAM, NVMe SSDs) to a single machine.

Why: Zero architectural complexity; code runs unmodified; zero network overhead between processes.

Trade-offs: Hard hardware ceiling; diminishing cost-to-performance ratio; single point of failure (SPOF) during hardware maintenance.

Horizontal Scaling (Scale-Out): Adding more commodity machines to form an interconnected pool.

Why: Virtually unlimited scale; high fault tolerance; resilient against isolated hardware crashes.

Trade-offs: Requires distributed coordination, data partitioning, network serialization latency, and stateless application designs.

1.2 Client-Server Communication Models

Short Polling: The client sends an HTTP request every N seconds.

Why: Dead simple to implement with standard REST.

Trade-offs: High server overhead, empty responses waste bandwidth, introduces latency bounded by polling interval.

Long Polling: The client requests data; the server holds the HTTP connection open until new data arrives or a timeout occurs. Once data arrives, the connection closes, and the client immediately opens another.

Why: Drastically reduces latency compared to short polling while still working over standard HTTP/1.1 proxies.

Trade-offs: Server connection thread starvation if thread-per-request models (like classic Tomcat) are used; reconnect storms during bulk updates.

WebSockets: Full-duplex, persistent TCP connection negotiated via an HTTP Upgrade handshake (`Connection: Upgrade`).

Why: Sub-millisecond bidirectional communication with minimal framing overhead (2–10 bytes per frame vs kilobytes of HTTP headers). Essential for live chat, stock tickers, and multiplayer synchronization.

Trade-offs: Stateful; complicates load balancing; requires sticky sessions or a pub/sub backplane (like Redis Pub/Sub) to route messages across instances.

Server-Sent Events (SSE): Persistent unidirectional HTTP streaming from server to client (`Content-Type: text/event-stream`).

Why: Built-in browser reconnection, lightweight, operates over standard HTTP/2 multiplexing. Ideal for LLM token streaming and live notifications.

Trade-offs: Unidirectional only (client cannot stream upstream over the same channel).

1.3 Communication Protocols: REST vs gRPC

Dimension	REST (Representational State Transfer)	gRPC (Google Remote Procedure Call)
Protocol	HTTP/1.1 or HTTP/2	Strict HTTP/2 (Multiplexed, bidirectional streaming)
Payload	JSON (Human-readable text)	Protocol Buffers (Compact binary serialization)
Contract	Loose (OpenAPI / Swagger optional)	Strict <code>.proto</code> schema compiled to native code

Dimension	REST (Representational State Transfer)	gRPC (Google Remote Procedure Call)
Performance	Higher parsing latency, payload bloat	High throughput, minimal CPU parsing overhead
Best Used For	External public APIs, web clients	Internal inter-service microservice RPCs

1.4 Load Balancing & Reverse Proxies

Layer 4 (Transport Layer): Operates on IP and TCP/UDP ports without inspecting packet payloads (e.g., AWS NLB, HAProxy TCP mode). Extremely fast, minimal CPU cost.

Layer 7 (Application Layer): Inspects HTTP headers, cookies, and URI paths (e.g., NGINX, Envoy, AWS ALB). Enables path-based routing (`/api/v1/orders` vs `/api/v1/users`), SSL termination, and rate limiting.

Load Balancing Algorithms:

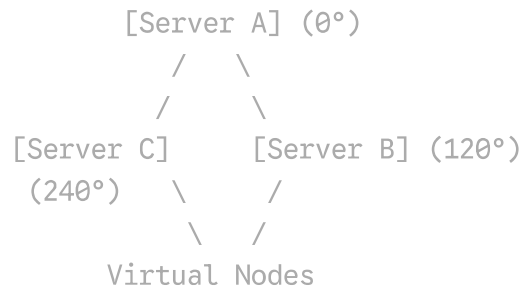
Round Robin / Weighted Round Robin: Distributes sequentially; fails when request execution time varies drastically.

Least Connections: Routes to the node with the fewest active TCP streams; ideal for long-lived sessions.

IP Hash / Consistent Hash: Routes identical client IPs to the same backend for session locality.

1.5 Consistent Hashing

In basic modular hashing ($Server = Hash(Key) \pmod N$), adding or removing a server invalidates almost 100% of keys, causing massive cache stampedes.



Mechanism: Both servers and data keys are mapped onto a 360-degree hash ring (e.g., using MD5 or MurmurHash3 to $[0, 2^{32} - 1]$). A key is assigned to the first server encountered moving clockwise.

Virtual Nodes (Vnodes): Each physical server is assigned multiple positions on the ring (e.g., 100–256 virtual tokens).

Why: Prevents hot-spots (non-uniform distribution) and ensures that when a server crashes, its load is distributed evenly across all remaining nodes rather than flooding a single neighbor.

1.6 Caching Strategies & Eviction Policies

Cache Read/Write Strategies:

Cache-Aside (Lazy Loading): Application checks cache; on miss, fetches from DB, updates cache, and returns. Application manages caching logic. DB and cache can drift if updates fail.

Read-Through: Application queries cache service directly. Cache service transparently fetches from DB on miss.

Write-Through: Application writes to cache; cache synchronously writes to DB before acknowledging. Strong consistency, high write latency.

Write-Back (Write-Behind): Application writes to cache; cache acknowledges immediately and batches asynchronous writes to DB. Ultra-low latency, risk of data loss on cache node crash.

Cache Eviction Algorithms:

LRU (Least Recently Used): Evicts items whose access time is oldest (implemented via Doubly Linked List + Hash Map).

LFU (Least Frequently Used): Evicts items with the lowest access count (implemented via Min-Heap or Frequency Linked Lists).

FIFO (First-In, First-Out): Evicts oldest inserted items regardless of usage.

1.7 Distributed Messaging: Apache Kafka Architecture

Kafka is a distributed, partitioned, replicated append-only commit log designed for high-throughput, fault-tolerant event streaming.





Topics & Partitions: A Topic is a logical category. Topics are split into Partitions, which are the physical unit of scalability and parallelism. Messages within a partition are strictly ordered by sequential IDs called Offsets.

Producers & Partitioning Keys: Producers publish events. If a message key is supplied, Kafka hashes it (`MurmurHash2(key) % num_partitions`) ensuring messages with the same key (e.g., `order_id`) always land on the exact same partition, guaranteeing strict per-entity ordering.

Brokers, Controllers, and ISR (In-Sync Replicas):

Each partition has one Leader broker and multiple Follower replicas.

Producers write exclusively to the Leader. Followers pull data to stay in sync.

ISR: Replicas that are fully caught up with the Leader. If a Leader crashes, the Kafka Controller elects a new Leader exclusively from the ISR pool.

Consumer Groups & Offset Management:

A Partition can be consumed by at most one consumer instance within a specific Consumer Group at any time. Adding consumers beyond the partition count leaves them idle.

Different consumer groups maintain independent offset pointers, enabling multiple systems (e.g., Billing, Inventory, Fraud) to process the same event stream

independently without duplicate work.

Delivery Semantics:

At-Most-Once: Commit offset before processing. If processing crashes, message is lost.

At-Least-Once: Process message, then commit offset. If crash occurs after processing but before commit, message is re-delivered. Requires idempotent consumer handlers.

Exactly-Once (EOS): Uses transactional producers (`sendOffsetsToTransaction`) coordinating commits across partitions and DB states via a two-phase commit.

1.8 Database Scaling: Replication, Partitioning, and CAP Theorem

Replication:

Leader-Follower (Master-Slave): Master handles all writes; slaves replicate logs and serve reads. Slaves can lag (Replication Lag), leading to stale reads.

Multi-Leader: Multiple nodes accept writes. High availability across regions, but requires conflict resolution strategies (Last-Write-Wins, CRDTs).

Leaderless (Dynamo-style): Quorum reads and writes ($R + W > N$).

Sharding Strategies:

Range-Based: Split by key ranges (e.g., IDs 1–10,000). Prone to hot spotting if keys increase sequentially.

Hash-Based: Hash key modulo shard count. Uniform distribution, but expensive to rebalance without consistent hashing.

CAP Theorem: In the presence of a Network Partition (P), a distributed system must choose between:

Consistency (C): Every read receives the most recent write or an error.

Availability (A): Every non-failing node returns a non-error response, without guaranteeing it contains the latest write.

PACELC Theorem: If there is a Partition (P), trade off Availability (A) and Consistency (C); Else (E), trade off Latency (L) and Consistency (C).

1.9 Distributed Transactions: 2PC vs Saga Pattern

Two-Phase Commit (2PC): A coordinator asks all participants to Prepare, and if all vote yes, asks them to Commit.

Why: Guarantees strong ACID across databases.

Trade-offs: Blocking protocol. If coordinator crashes during the commit phase, locks are held indefinitely, destroying system availability.

Saga Pattern: A sequence of local transactions where each updates its local database and publishes an event triggering the next step.

Compensating Transactions: If a step fails, the saga runs compensating actions backward (e.g., refund money if inventory reservation fails).

Orchestration: A central Saga Orchestrator tells participants what local transaction to execute next.

Choreography: Each service listens to events and decides when to trigger its local transaction.

2. Advanced Tech Deep Dives (From Your Profile & Stack)

2.1 Spring Boot, Spring Security & JWT Architecture

Spring Boot Internal Architecture: Operates around the `DispatcherServlet`, which catches incoming HTTP requests, queries `HandlerMapping`, invokes the matching `@RestController`, and processes serialization via `HttpMessageConverter` (Jackson) before passing through Filters and Interceptors.

Spring Security Filter Chain: Security is enforced via a chain of servlet filters (`SecurityFilterChain`).

Incoming requests pass through `CorsFilter`, `CsrfFilter`, and custom authentication filters before hitting business controllers.

JWT (JSON Web Token) Lifecycle:

Structure: `Header.Payload.Signature` encoded in Base64URL.

Signature Verification: Uses HMAC-SHA256 (`HS256`) or RSA (`RS256`):

$$\text{Signature} = \text{HMAC-SHA256}(\text{base64Url}(\text{header}) + "." + \text{base64Url}(\text{payload}))$$

Stateless Authentication Flow: Client passes JWT in

Authorization: `Bearer <token>`. A custom `JwtAuthenticationFilter` intercepts the request, verifies the signature against the server's public key/secret, extracts the claims (`sub`, `roles`), and injects an `Authentication` object into the `SecurityContextHolder`. No database session query required.

2.2 Relational vs Non-Relational & PostgreSQL JSONB

Relational Storage Engine (PostgreSQL / MySQL InnoDB):

Employs B+Trees for indexed lookups ($O(\log N)$). Enforces strict ACID using Write-Ahead Logging (WAL) for durability and Multi-Version Concurrency Control (MVCC) for non-blocking reads.

PostgreSQL JSONB Internals:

Unlike raw `JSON` (which stores exact text representations), `JSONB` stores decomposed binary formats. It parses whitespace, normalizes keys, and supports GIN (Generalized Inverted Index) indexing.

Use Case in NestXchange: Instead of creating separate database tables for different listing categories (properties, vehicles), a universal `Listing` table holds standard relational columns (`id` , `price` , `user_id` , `status`), while dynamic attributes (e.g., `mileage` , `bedrooms` , `furnishing`) reside inside a single `JSONB` column. GIN indexing over this column enables high-speed filtered queries without dynamic schema alterations.

2.3 Redis Internals & Concurrency Control

In-Memory Architecture: Uses an event-driven single-threaded execution loop (I/O Multiplexing via `epoll`) to achieve sub-millisecond latencies across hundreds of thousands of operations per second, eliminating thread context-switching overhead and locking bottlenecks.

Distributed Locking (Redlock / SETNX):

To prevent double-booking or race conditions across microservice clusters:

Bash



```
SET resource_key my_unique_uuid NX PX 30000
```

`NX` : Set only if key does not exist.

PX 30000 : Set automatic expiry to 30,000 milliseconds to avoid deadlocks if the holder dies.

Releasing the Lock: Must be executed via an atomic Lua script that compares the stored UUID with the caller's UUID before deleting, ensuring an instance does not accidentally unlock a resource whose lock expired and was re-acquired by another process.

2.4 Agentic LLM Architectures: Planner-Executor-Reflector

Planner: Takes a complex goal, queries domain metadata, and decomposes the request into an ordered Directed Acyclic Graph (DAG) of discrete operational tasks.

Executor: Executes individual tasks by invoking specific external tools, database queries, or microservice REST/gRPC endpoints using structured JSON function calling.

Reflector (Evaluator): Assesses the execution output against expected preconditions and validation schemas. If an error or degraded result occurs, it feeds the execution trajectory and error log back to the Planner for self-correction.

Risk-Tiered Human-in-the-Loop (HITL): Actions categorized by financial or operational risk:

Tier 1 (Read-Only / Low-Risk): Automated execution (e.g., checking inventory levels, reading order status).

Tier 2 (Medium-Risk): Automated execution with audit notifications (e.g., updating delivery addresses).

Tier 3 (High-Risk): Execution paused; state persisted in PostgreSQL; an approval ticket is dispatched to an operations dashboard. Resumption occurs only upon valid cryptographic employee sign-off.

2.5 Retrieval-Augmented Generation (RAG) Architecture

Ingestion Pipeline:

1. *Document Parsing*: Extract clean text from unstructured sources (PDFs, Markdown, Web pages) using tools like Apache Tika.
2. *Chunking*: Split text using recursive character splitters (500–1000 tokens) with a sliding overlap window (10–20%) to preserve context across boundaries.
3. *Embedding Generation*: Map textual chunks into high-dimensional dense vector spaces (e.g., 1536 dimensions) using models like OpenAI `text-embedding-3-small` or open-source HuggingFace models.
4. *Vector Database Indexing*: Store embeddings in vector stores (ChromaDB, Pinecone, pgvector) using Approximate Nearest Neighbor (ANN) index algorithms:
 - ↳ *HNSW (Hierarchical Navigable Small World)*: Multi-layered graphs offering high query throughput and recall at the expense of memory.
 - ↳ *IVF (Inverted File Index)*: Clusters vector space into Voronoi cells; narrows search to closest centroids.

Query & Retrieval Pipeline:

1. User query is transformed into a dense embedding using the same embedding model.
2. Perform cosine similarity or dot product search across the vector index to retrieve the top- K most relevant text chunks:

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

3. *Re-Ranking (Cross-Encoder)*: Pass the top- K candidates through a fine-grained re-ranking model to compute exact relevance scores, pruning irrelevant results.

4. *Context Injection*: Augment the original prompt with retrieved chunks inside strict delimiting XML tags and instruct the LLM to ground its response exclusively within the provided context.

2.6 Computer Vision & ANPR Pipeline

YOLO (You Only Look Once): Single-stage object detector framing detection as a regression problem. It divides the input image into an $S \times S$ grid. Each grid cell predicts B bounding boxes, confidence scores, and class conditional probabilities in a single forward pass, achieving real-time inference (30–60 FPS).

Image Preprocessing Pipeline (OpenCV):

1. *Grayscale Conversion*: Eliminates redundant chromatic channels, reducing processing overhead.
2. *Bilateral Filtering / Gaussian Blur*: Removes high-frequency camera noise while preserving sharp plate edge boundaries.
3. *Adaptive Thresholding (Otsu's / Sauvola)*: Converts grayscale images to binary (pure black and white) by calculating localized threshold values across neighborhoods, compensating for strong shadows and headlight glare.
4. *Morphological Transformations*: Applies dilation and erosion to close character gaps and isolate character contours.

OCR Engines (PaddleOCR / EasyOCR):

Uses a two-stage approach: a Text Detection Network (DBNet) locates character bounding polygons, and a Text Recognition Network (CRNN with CTC loss or SVTR) decodes character sequences.

3. High-Level Design (HLD) Solutions: 8 Comprehensive Architectures

System 1: Netflix (Global Video Streaming System)

Requirements

Functional: Upload/encode 4K videos; globally stream video with adaptive bitrate; personalized home feed; track playback progress.

Non-Functional: High availability (99.999%); low playback startup latency (< 500 ms); massive read-to-write ratio (1000 : 1); fault-tolerant global edge delivery.

Scale & Estimations

Daily Active Users (DAU): 100 Million.

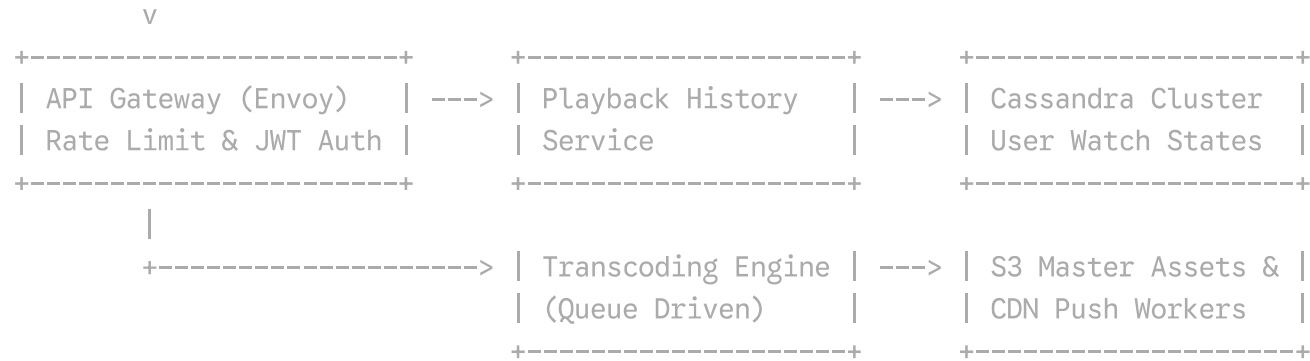
Concurrent streams at peak: 15 Million.

Average video bitrate: 3 Mbps (Adaptive 500 kbps to 15 Mbps).

Peak Outbound Bandwidth: 15 Million $\times$ 3 Mbps = 45 Tbps (Served entirely via CDN / Open Connect Appliances).

Architecture & Component Diagram





Transcoding & Storage Pipeline

1. Content teams upload raw master mezzanine video files to an Amazon S3 staging bucket.
2. An S3 `ObjectCreated` event drops an event into an Apache Kafka queue.
3. Transcoding workers pull jobs, chunking the video into 2- to 10-second segments.
4. Each segment is transcoded into multiple codecs (H.264/AVC, H.265/HEVC, AV1) across dozens of resolutions (360p, 720p, 1080p, 4K) and bitrates.
5. Transcoding workers generate manifest files:

HLS (HTTP Live Streaming): Master `.m3u8` playlist referencing resolution-specific playlists.

DASH (Dynamic Adaptive Streaming over HTTP): `.mpd` XML manifest.

6. Encoded chunks are distributed across Netflix Open Connect Appliances (OCAs)—custom FreeBSD storage servers embedded directly inside regional ISPs worldwide.

Database Strategy & Data Flow

Cassandra (Playback State & Metadata):

Why: Cassandra's masterless ring architecture handles massive write throughput with linear horizontal scale.

Schema:

SQL



```
CREATE TABLE user_playback_progress (  
    user_id uuid,  
    profile_id uuid,  
    video_id uuid,  
    offset_seconds int,  
    completed_percentage float,  
    last_updated timestamp,  
    PRIMARY KEY ((user_id, profile_id), video_id)  
);
```

Partition key `(user_id, profile_id)` groups all watch history for a single user onto the same partition node, enabling single-digit millisecond reads when rendering the "Continue Watching" carousel.

System 2: Uber / Grab (Real-Time Location & Ride Matching)

Requirements

Functional: Real-time driver location updates every 4 seconds; rider requests pickup; match rider to nearby active drivers within a radius; live tracking during transit.

Non-Functional: Sub-second latency for location ingestion; high availability over consistency for location pings; fault-tolerant driver-to-rider matching.

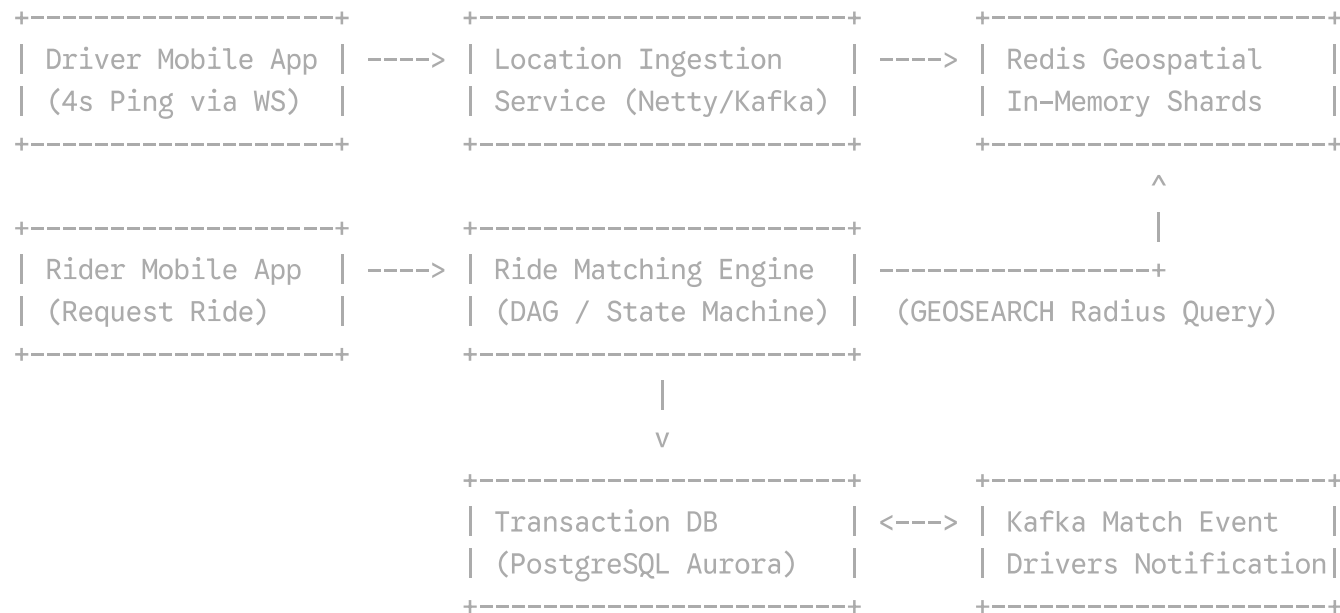
Scale & Estimations

Active Drivers: 1 Million concurrent.

Active Riders: 10 Million.

Ingestion Rate: 1 Million updates/4 sec = 250,000 QPS location write throughput.

Architecture & Component Diagram



Geospatial Indexing Strategy: H3 vs S2 vs Geohash

Geohash: Interleaves latitude/longitude into base32 strings forming hierarchical rectangular bounding boxes. Edge distortion occurs near poles and meridian boundaries.

Google S2: Projects Earth onto a cube with Hilbert space-filling curves. Quadrilateral cells, strong performance, but non-uniform edge-to-center distances.

Uber H3 (Hexagonal Hierarchical Spatial Index):

Why Chosen: Hexagons have invariant neighbor distances: every adjacent cell center is equidistant (unlike squares where diagonal neighbors are $\sqrt{2}$ times farther). This makes radius searches, demand-heat calculations, and surge-pricing cluster expansions mathematically uniform.

Spatial resolution 8 (average hexagon area $\approx 0.737 \text{ km}^2$, edge length ≈ 461 meters) perfectly matches localized driver search radiuses.

Location Ingestion Flow & Redis Sharding

1. Drivers maintain persistent WebSocket connections to a pool of Netty-based Gateway servers.
2. Every 4 seconds, the driver transmits `{driver_id, lat, lng, status: "AVAILABLE"}`.
3. The Gateway pushes location pings to an Apache Kafka `driver-locations` topic, partitioned by `H3Index(lat, lng, res_8)`.
4. Consumer workers consume from Kafka and update a cluster of Redis instances:

Bash



```
GEOADD drivers:res8:<h3_parent_index> <lng> <lat> <driver_id>
```

5. Redis structures internal keys using Sorted Sets (`ZSET`), where the score is a 52-bit integer encoding the spatial coordinate (Geohash-derived integer).
6. When a rider requests a pickup:

The Matching Engine converts the rider coordinate to an H3 index.

It queries Redis using `GEOSEARCH` across the rider's cell and its 6 immediate neighboring hexagonal rings:

Bash



```
GEOSEARCH drivers:res8:<h3_index> FROMLONLAT <rider_lng> <rider_lat> BYRADIUS 3
```



The top candidate drivers are dispatched to a Kafka `dispatch-offers` topic.

System 3: Food Delivery Platform (DoorDash / Zomato / Swiggy)

Requirements

Functional: Dynamic restaurant menu rendering; cart checkouts; real-time transactional order placement; multi-role tracking (Customer, Restaurant, Delivery Partner); asynchronous status notifications.

Non-Functional: Zero lost orders (Strict ACID transactional boundaries); sub-second checkout latency; resilient under meal-time spikes ($5 \times$ base traffic).

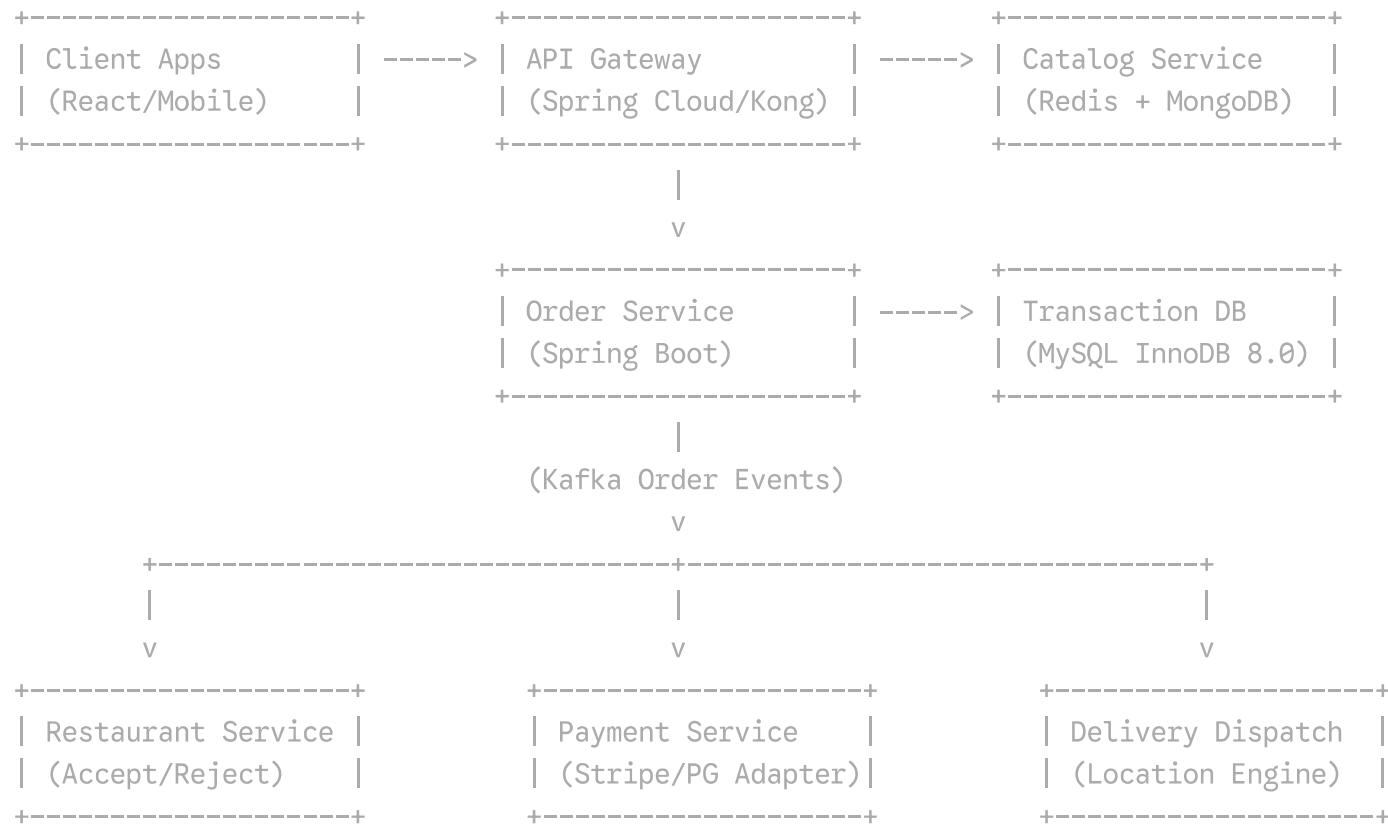
Scale & Estimations

Daily Orders: 5 Million orders/day.

Peak Order Throughput: 2,500 orders/second.

Average order latency requirement: < 800 ms for checkout authorization.

Architecture & Microservices Topology



Transactional Order Placement & Concurrency

Preventing Race Conditions (The "Last Item" Problem): When thousands of users attempt to purchase a limited menu promotion simultaneously, a standard `SELECT` followed by an `UPDATE` leads to double-allocation due to lost updates.

Concurrency Mechanisms Evaluated:

Optimistic Locking: Reads row with a `version` column. Updates only if `WHERE version = :read_version`. Fails high-concurrency requests with high rollback retry overhead.

Pessimistic Locking: Uses `SELECT ... FOR UPDATE`. Locks the row at the database engine level; serializes transactions cleanly, but creates lock contention bottlenecks under heavy load.

Distributed Redis Token Pre-Allocation: Decrement inventory atomically in Redis using an atomic script:

Lua



```
if redis.call("EXISTS", KEYS[1]) == 1 then
    local stock = tonumber(redis.call("GET", KEYS[1]))
    if stock > 0 then
        redis.call("DECRBY", KEYS[1], 1)
        return 1
    else
        return 0
    end
else
    return -1
end
```

Once Redis successfully decrements the token, the request asynchronously commits to the MySQL database.

Order Lifecycle State Machine & The Saga Pattern

Order States: `CREATED` → `PAYMENT_PENDING` → `PAID` → `RESTAURANT_ACCEPTED` → `PREPARING` → `OUT_FOR_DELIVERY` → `DELIVERED` → `REFUNDED` .

An Order Orchestrator coordinates steps using Kafka:

1. *OrderService* inserts order with status `CREATED` .
2. *PaymentService* executes payment. If failed, transitions order to `CANCELLED` .
3. *RestaurantService* validates kitchen queue. If rejected, triggers compensating step: *PaymentService* issues a `REFUND` .
4. If accepted, dispatches message to *DeliveryDispatchService* to locate courier.

System 4: WhatsApp / Telegram (End-to-End Encrypted Real-Time Chat)

Requirements

Functional: 1-on-1 private messaging; group chats (1000+ members); delivery receipts (Sent, Delivered, Read); offline message queuing; media attachment handling.

Non-Functional: Sub-200 ms message latency globally; End-to-End Encryption (E2EE); zero persistent plain-text storage on application servers; minimal mobile battery consumption.

Scale & Estimations

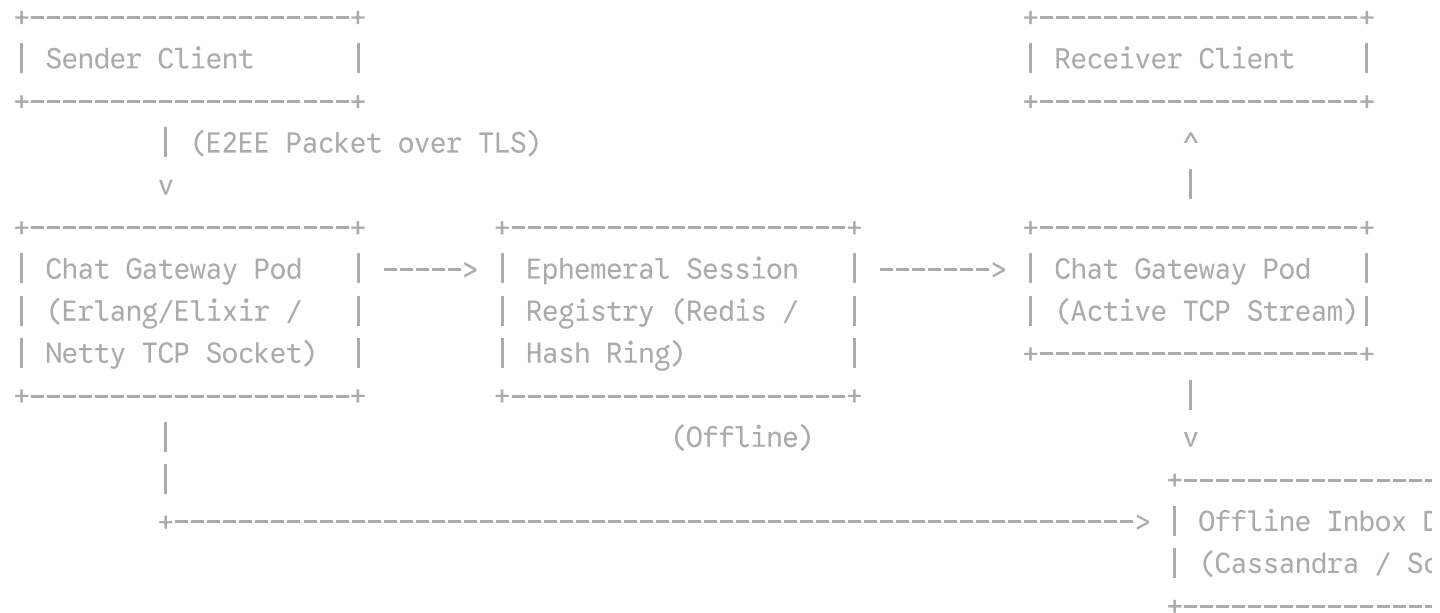
Active Users: 2 Billion.

Messages per day: 100 Billion.

Average message QPS: 1, 150, 000 msgs/sec.

Peak QPS: 3, 000, 000 msgs/sec.

Architecture & Component Diagram



Protocol Selection: XMPP vs MQTT vs Raw WebSockets

XMPP: Heavy XML framing bloats mobile cellular data; high CPU overhead for parsing.

MQTT: Lightweight binary publish/subscribe protocol designed for constrained IoT networks; excellent for battery efficiency, but lacks custom application-level session routing semantics required for chat.

Custom Protocol over TLS/TCP (Noise Protocol + WebSockets): WhatsApp uses a customized variant of XMPP over noise protocol framing, while Telegram uses MTProto. A lightweight binary protobuf framing over persistent TLS TCP sockets minimizes header overhead to bytes.

E2EE (Signal Protocol) Details

Utilizes the Double Ratchet Algorithm combining:

1. Extended Triple Diffie-Hellman (X3DH) key exchange for mutual authentication and initial shared secret derivation when starting sessions.
2. Symmetric-key Ratchet: Generates new ephemeral encryption keys for every single message sent. Even if a private key is compromised, historical messages remain cryptographically protected (Forward Secrecy).

Application servers route encrypted opaque payloads: the server sees only sender ID, receiver ID, and binary ciphertext.

Connection Maintenance & Offline Queuing

1. Each client maintains an active bidirectional socket with a Chat Gateway instance.
2. The Gateway maps `user_id` -> `gateway_instance_ip` in an in-memory Redis cluster.
3. When User A messages User B:

The Gateway looks up User B's current connected gateway IP in Redis.

If User B is online, the message is routed directly through an internal cross-node TCP connection to User B's gateway and delivered over the socket.

User B acknowledges receipt; the gateway sends a "Delivered" ACK back to User A.

4. If User B is offline:

The message is appended to User B's inbox table in ScyllaDB/Cassandra.

An Apple APNs or Google FCM Push Notification is triggered.

Once User B reconnects, their gateway pulls unread rows sequentially, streams them down the socket, and deletes them from ScyllaDB upon confirmation.

System 5: Twitter / X (Timeline Generation & Fanout Engine)

Requirements

Functional: Post tweets (280 characters, media links); follow users; render home timeline (tweets from followed users, chronologically or ranked); render user profile timeline.

Non-Functional: Sub-100 ms timeline fetch latency; fast write ingestion; scalable under extreme fanout conditions (celebrity accounts with 100M+ followers).

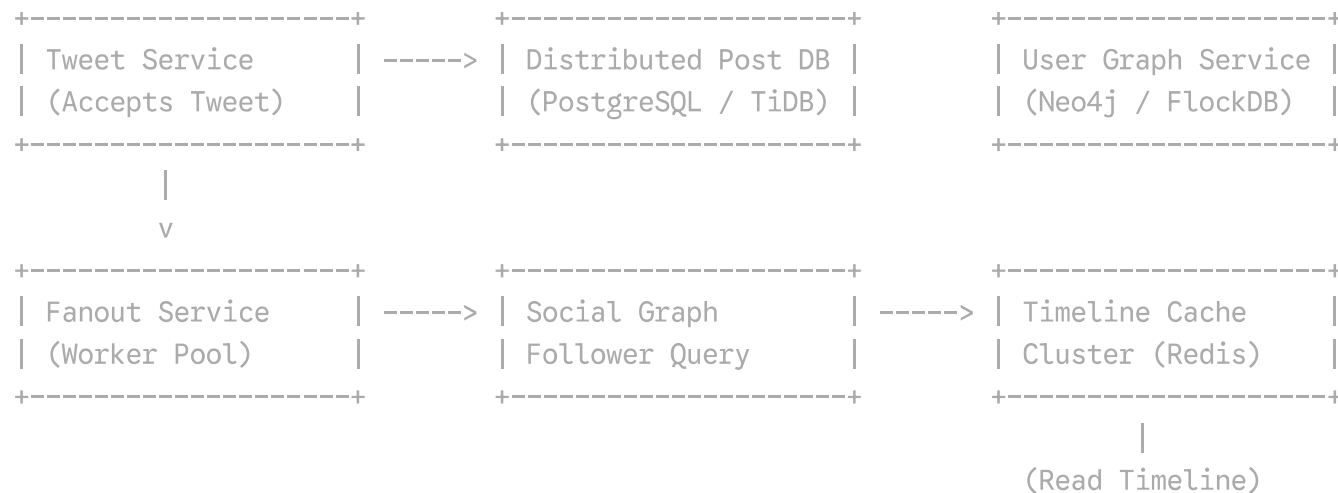
Scale & Estimations

Daily Active Users (DAU): 300 Million.

Tweet Post QPS: 6,000 writes/second (Peak: 25,000/sec).

Timeline Read QPS: 300,000 reads/second (Read-heavy: 50 : 1).

Architecture & Component Diagram



v

```
+-----+
| Home Timeline Svc |
| (Client API)      |
+-----+
```

Fanout-on-Write (Push) vs Fanout-on-Read (Pull)

Fanout-on-Write (Push Model):

When a user posts a tweet, the Fanout Service queries the Social Graph for all followers.

It injects the `tweet_id` into the Redis home timeline list of every follower:

Bash



```
LPUSH timeline:user:<follower_id> <tweet_id>
LTRIM timeline:user:<follower_id> 0 799 # Keep latest 800 tweets
```

Advantage: Timeline read is instantaneous ($O(1)$ lookup via

```
LRANGE timeline:user:<id> 0 20 ).
```

Failure Mode (The Celebrity Problem): When an account with 100 Million followers posts, the system must perform 100 Million Redis writes, creating severe lag and memory exhaustion.

Fanout-on-Read (Pull Model):

Tweets are stored only in the author's tweet table.

When a user loads their timeline, the system fetches the list of accounts they follow, queries their latest tweets, and merges them using a K -way merge algorithm in memory.

Advantage: Zero write latency.

Disadvantage: High read latency; queries become exponentially slower as follow counts scale.

Hybrid Architecture (The Industry Standard):

Use **Fanout-on-Write** for regular users (followers $< 25,000$).

Use **Fanout-on-Read** for celebrities (followers $\geq 25,000$).

When a follower loads their timeline:

1. Read their precomputed Redis timeline (populated by regular users).
2. Query the latest tweets of the few celebrities they follow.
3. Merge the lists in memory sorted by timestamp.

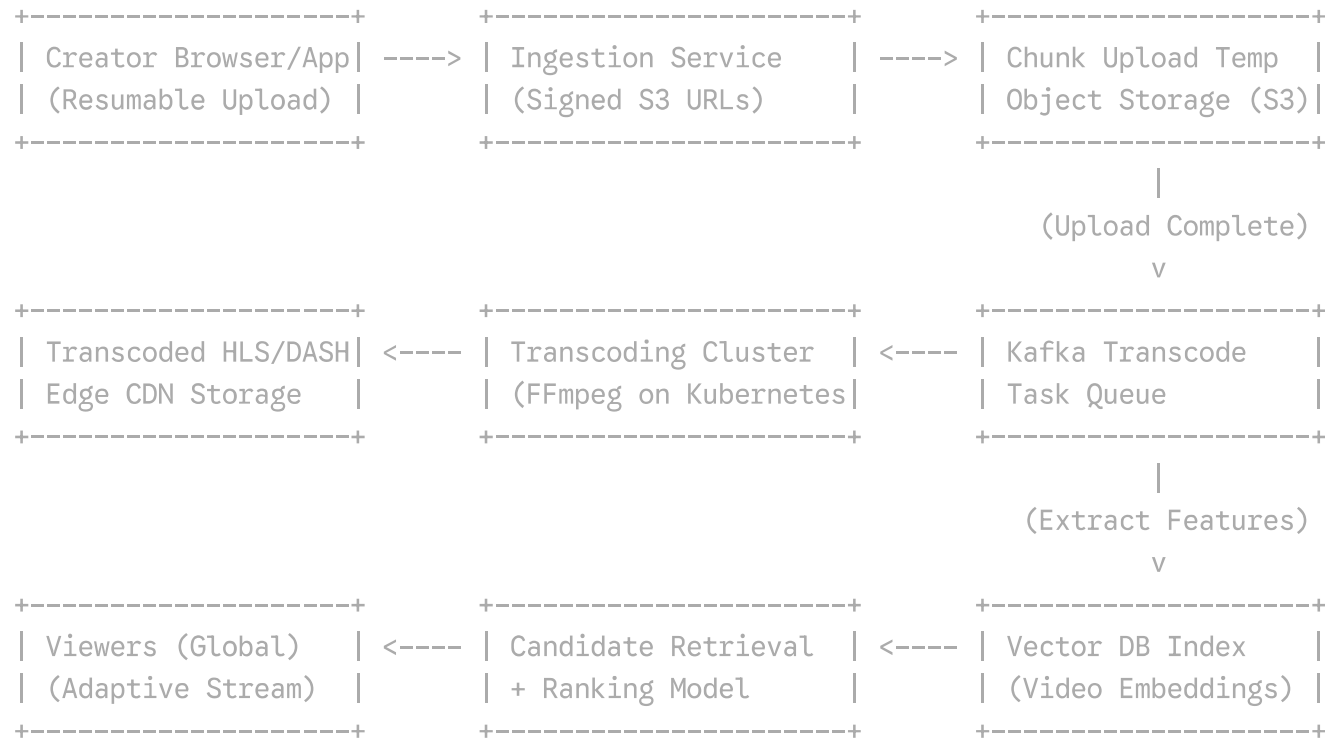
System 6: YouTube / TikTok (Video Transcoding, Recommendation & Serving)

Requirements

Functional: Upload videos up to several gigabytes; automated transcoding into adaptive profiles; global streaming playback; comment & like interactions; recommendation engine feed.

Non-Functional: Zero corruption of uploaded files; minimal buffering ($< 1\%$ session time); highly resilient transcoding pipeline.

Architecture & Component Diagram



Chunked Resumable Upload Flow

1. Client initiates upload: `POST /api/v1/videos/uploads/initialize`.
2. Upload Service creates an upload session ID and returns a sequence of presigned Amazon S3 multipart upload URLs.
3. Client divides video into 5MB chunks and transmits them via parallel `PUT` requests with content-range headers. If chunk 12 fails due to network drop, the client retries only chunk 12.

4. Once all chunks succeed, client triggers `POST /api/v1/videos/uploads/complete`. S3 joins parts into a unified master binary.

Distributed Transcoding Pipeline

1. Ingestion triggers a message to Kafka containing the S3 path of the raw master video.
2. A Master Transcoder worker pulls the video, inspects metadata using FFmpeg, and splits the file into 5-second segments (`.ts` or fragmented `.mp4`).
3. Kubernetes autoscaling worker pods pull segment batches and transcode in parallel:

Bash



```
ffmpeg -i segment_001.ts -vf scale=1920:1080 -c:v libx264 -b:v 4500k -c:a aac output_
ffmpeg -i segment_001.ts -vf scale=1280:720 -c:v libx264 -b:v 2200k -c:a aac output_
```



4. Workers compile individual playlist manifests (`index_1080p.m3u8` , `index_720p.m3u8`) and a master `master.m3u8` manifest file.
5. Files are pushed to persistent object storage and primed across Edge CDN pops.

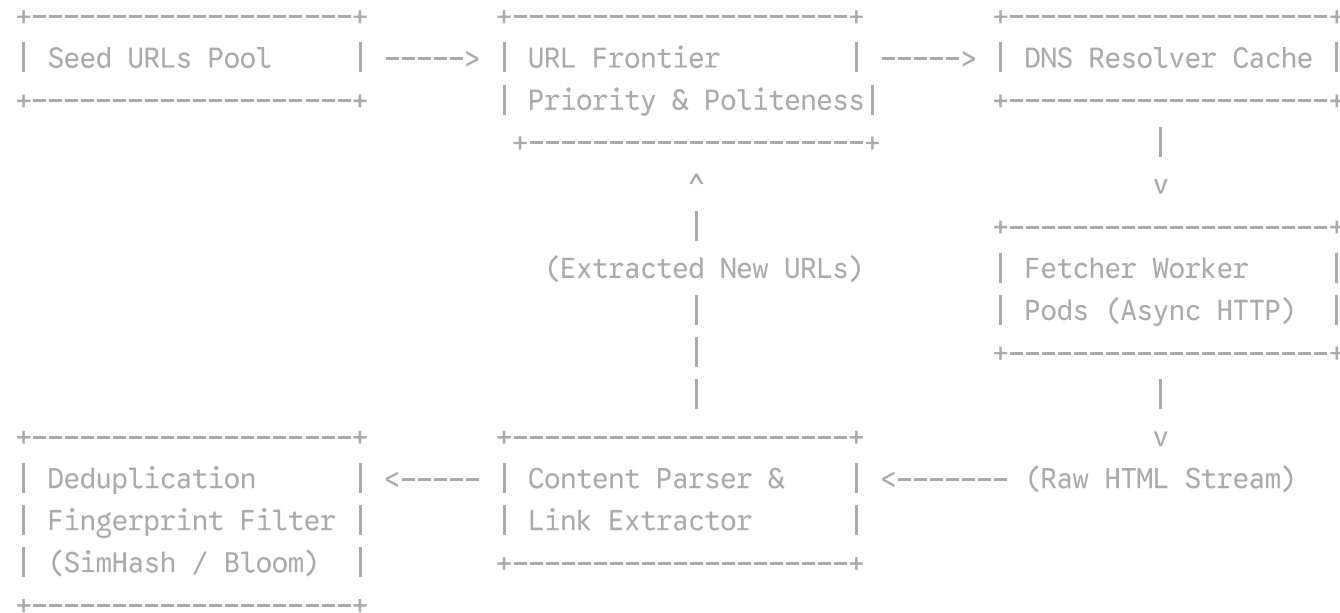
System 7: Distributed Web Crawler (Google / Common Crawl Scale)

Requirements

Functional: Ingest seed URLs; recursively extract hyperlinks; fetch HTML; parse content; index metadata; handle deduplication; respect robot exclusion standards (`robots.txt`).

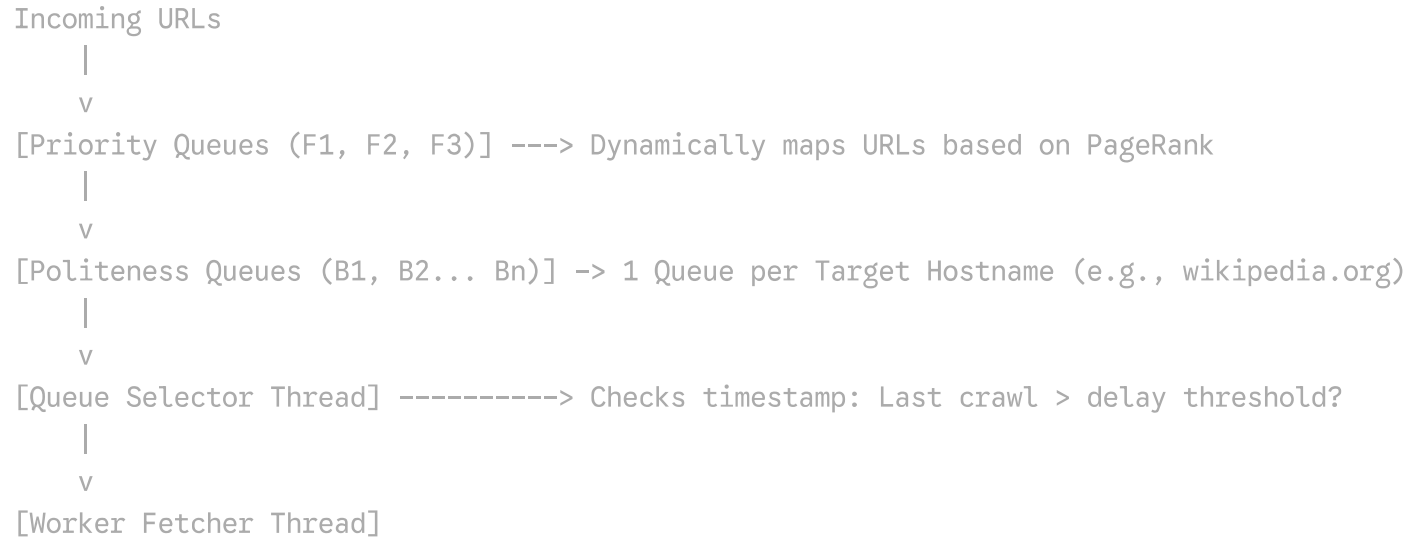
Non-Functional: Politeness (do not overwhelm target hosts); crawl throughput exceeding 50,000 pages/sec; fault tolerance across network partitions; duplicate document detection.

Architecture & Component Diagram



URL Frontier Architecture: Politeness & Priority

A naive FIFO queue crawls identical domains consecutively, effectively launching an accidental Distributed Denial of Service (DDoS) against websites.



1. **Priority Queues (Front Queues):** Categorizes URLs based on domain importance, PageRank score, and freshness needs.
2. **Politeness Queues (Back Queues):** Exactly one queue per target host name (e.g., `queue:nytimes.com` , `queue:github.com`).
3. Each queue has an associated state holding the timestamp of the last request sent to that host.
4. A round-robin worker selector checks queues: it dequeues a URL only if `currentTime - lastAccessTime >= CrawlDelay` (parsed from `robots.txt`).

Deduplication at Scale: Bloom Filters & SimHash

URL Deduplication: Checking whether a URL was already visited cannot query a database at 50,000 QPS. We use a **Bloom Filter** in memory:

A bit array of size m with k independent hash functions.

Zero false negatives: If it returns false, the URL has definitely never been crawled.

Tolerates minute false positives: A tiny fraction of new URLs might be skipped, which is acceptable at scale.

Content Deduplication (Near-Duplicate Webpages): Mirrors, scrapers, and dynamic page wrappers contain identical textual content under different URLs.

SimHash Algorithm:

1. Extract features (words) from the HTML text and assign weights based on frequency.
2. Hash each word into an n -bit fingerprint ($n = 64$).
3. Sum bit positions: for bit position i , if the feature hash has bit 1, add the weight; if bit 0, subtract the weight.
4. Construct a 64-bit vector: if sum > 0 , set bit to 1; else set to 0.
5. Two web documents are near-duplicates if the Hamming distance between their SimHash tokens is ≤ 3 .

System 8: Distributed Rate Limiter (API Security & Token Engine)

Requirements

Functional: Limit API client access based on API token, IP, or route; support tiered limits (e.g., 100 req/min for Free, 10,000 req/min for Enterprise); return HTTP 429 (Too Many Requests) with retry headers.

Non-Functional: Extremely low latency impact (< 2 ms processing overhead per request); zero memory leaks; resilient against distributed clock drift.

Comparison of Rate Limiting Algorithms

Token Bucket: Tokens are continuously added to a bucket at a fixed fill rate up to capacity. Each request consumes a token. Allows controlled traffic bursts up to bucket capacity.

Leaking Bucket: Requests enter a FIFO queue and leak out at a constant execution rate. Smooths out traffic spikes into steady processing; drops requests when queue is full.

Fixed Window Counter: Divides time into fixed intervals (e.g., 1 minute). Counts requests per window.

Vulnerability: A traffic burst at the boundary edge (e.g., last 5 seconds of Minute 1 and first 5 seconds of Minute 2) can allow double the limit within a 10-second window.

Sliding Window Log: Stores an exact timestamp for every request in a sorted set. Accurate, but memory footprint explodes under high traffic.

Sliding Window Counter: Approximates sliding window counts mathematically by combining the previous window's count with the current window:

$$\text{Count} = \text{Count}_{\text{current}} + \left(\text{Count}_{\text{previous}} \times \left(1 - \frac{\text{Current Window Offset}}{\text{Window Size}} \right) \right)$$

Provides high accuracy with constant $O(1)$ memory usage.

Distributed Implementation: Redis + Atomic Lua Script

To prevent race conditions where concurrent microservice nodes read identical counters simultaneously, execution is unified inside an atomic Lua script:

Lua



```
local key = KEYS[1]
local limit = tonumber(ARGV[1])
local current_time = tonumber(ARGV[2])
local window = tonumber(ARGV[3])
local clear_before = current_time - window

-- Remove timestamps older than the sliding window threshold
redis.call('ZREMRANGEBYSCORE', key, 0, clear_before)

-- Count remaining requests in the active window
local current_requests = redis.call('ZCARD', key)

if current_requests < limit then
    -- Add unique member (current_time:unique_random_id)
    redis.call('ZADD', key, current_time, current_time .. ":" .. redis.call('TIME')[2])
    redis.call('EXPIRE', key, window)
    return 1 -- Request Allowed
else
    return 0 -- Request Rate Limited
end
```

4. Low-Level Design (LLD) & Object-Oriented Patterns: 7 Concrete Implementations

System 9: Distributed URL Shortener (TinyURL)

Core Logic: Base62 vs Hashing

Using an MD5 or SHA-256 hash yields 128/256 bits, which requires truncation. Truncation increases collision probability, requiring expensive database collision lookups.

Base62 Encoding of Sequential / Snowflake IDs:

Characters: $[a-z, A-Z, 0-9] = 62$ characters.

A 7-character Base62 string represents:

$$62^7 \approx 3.52 \text{ Trillion unique URLs}$$

Approach: Take a unique 64-bit monotonically increasing integer from a distributed ID generator (or range allocator) and convert it directly into Base62.

Java Implementation

Java



```
package com.design.urlshortener;

import java.time.Instant;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

public class TinyUrlSystem {

    private static final String BASE62_CHARS = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    private static final int BASE = BASE62_CHARS.length();

    // Range-allocated counter simulating unique cluster-generated IDs
    private final AtomicLong globalCounter = new AtomicLong(1000000000L);
```

```
// In-memory persistent caches for demo purposes
private final ConcurrentHashMap<String, UrlMapping> shortToLongMap = new ConcurrentHashMap<>();
private final ConcurrentHashMap<String, String> longToShortMap = new ConcurrentHashMap<>();

public record UrlMapping(String longUrl, String shortCode, Instant createdAt, Instant expiresAt) {}

public String shortenUrl(String longUrl, long ttlSeconds) {
    if (longToShortMap.containsKey(longUrl)) {
        return longToShortMap.get(longUrl);
    }

    long uniqueId = globalCounter.incrementAndGet();
    String shortCode = encodeBase62(uniqueId);

    Instant now = Instant.now();
    Instant expires = (ttlSeconds > 0) ? now.plusSeconds(ttlSeconds) : Instant.MAX;

    UrlMapping mapping = new UrlMapping(longUrl, shortCode, now, expires);
    shortToLongMap.put(shortCode, mapping);
    longToShortMap.put(longUrl, shortCode);

    return shortCode;
}

public String getOriginalUrl(String shortCode) {
    UrlMapping mapping = shortToLongMap.get(shortCode);
    if (mapping == null) {
        throw new IllegalArgumentException("Short URL does not exist.");
    }
    if (Instant.now().isAfter(mapping.expiresAt())) {
        shortToLongMap.remove(shortCode);
        longToShortMap.remove(mapping.longUrl());
        throw new IllegalStateException("Short URL has expired.");
    }
}
```

```

        return mapping.longUrl();
    }

    private String encodeBase62(long value) {
        StringBuilder sb = new StringBuilder();
        while (value > 0) {
            int remainder = (int) (value % BASE);
            sb.append(BASE62_CHARS.charAt(remainder));
            value /= BASE;
        }
        return sb.reverse().toString();
    }
}

```

System 10: In-Memory LRU Cache with TTL (Thread-Safe)

Design Rationale & Complexity

Hash Map: Provides $O(1)$ key-to-node lookup.

Doubly Linked List: Enables $O(1)$ node removal and insertion at the head.

ReentrantReadWriteLock: Allows concurrent readers while serializing write updates, preventing data races under multithreaded loads.

```

      +-----+      +-----+      +-----+      +-----+
HEAD <-> Node | <-> | Node  | <-> | Node  | <-> | Tail |

```

+-----+
(Most Recent)

+-----+

+-----+

+-----+
(Least Recent)

Complete Implementation

Java



```
package com.design.lrucache;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class ThreadSafeLruCache<K, V> {

    private static class Node<K, V> {
        K key;
        V value;
        long expirationTimeMs;
        Node<K, V> prev;
        Node<K, V> next;

        Node(K key, V value, long expirationTimeMs) {
            this.key = key;
            this.value = value;
            this.expirationTimeMs = expirationTimeMs;
        }

        boolean isExpired() {
            return expirationTimeMs > 0 && System.currentTimeMillis() > expirationTimeMs;
        }
    }
}
```

```
    }  
}  
  
private final int capacity;  
private final Map<K, Node<K, V>> map;  
private final Node<K, V> head;  
private final Node<K, V> tail;  
private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();  
  
public ThreadSafeLruCache(int capacity) {  
    this.capacity = capacity;  
    this.map = new HashMap<>();  
    this.head = new Node<>(null, null, 0);  
    this.tail = new Node<>(null, null, 0);  
    head.next = tail;  
    tail.prev = head;  
}  
  
public V get(K key) {  
    lock.writeLock().lock();  
    try {  
        Node<K, V> node = map.get(key);  
        if (node == null) return null;  
  
        if (node.isExpired()) {  
            removeNode(node);  
            map.remove(key);  
            return null;  
        }  
  
        moveToHead(node);  
        return node.value;  
    } finally {  
        lock.writeLock().unlock();  
    }  
}
```

```
}
```

```
public void put(K key, V value, long ttlMs) {  
    lock.writeLock().lock();  
    try {  
        long expireAt = (ttlMs > 0) ? System.currentTimeMillis() + ttlMs : 0;  
        Node<K, V> existingNode = map.get(key);  
  
        if (existingNode != null) {  
            existingNode.value = value;  
            existingNode.expirationTimeMs = expireAt;  
            moveToHead(existingNode);  
        } else {  
            if (map.size() >= capacity) {  
                Node<K, V> lru = tail.prev;  
                removeNode(lru);  
                map.remove(lru.key);  
            }  
            Node<K, V> newNode = new Node<>(key, value, expireAt);  
            addNodeToHead(newNode);  
            map.put(key, newNode);  
        }  
    } finally {  
        lock.writeLock().unlock();  
    }  
}
```

```
private void addNodeToHead(Node<K, V> node) {  
    node.next = head.next;  
    node.prev = head;  
    head.next.prev = node;  
    head.next = node;  
}
```

```
private void removeNode(Node<K, V> node) {
```

```

        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void moveToHead(Node<K, V> node) {
        removeNode(node);
        addNodeToHead(node);
    }
}

```

System 11: Elevator Control System (Dispatcher & Scheduling)

Design Patterns Applied

State Pattern: Encapsulates elevator states (`MOVING_UP` , `MOVING_DOWN` , `IDLE` , `MAINTENANCE`).

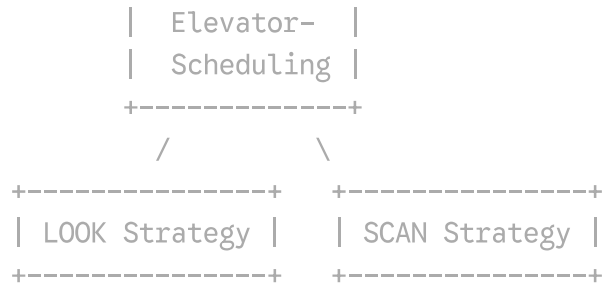
Strategy Pattern: Decouples the scheduling algorithm (`LOOK` , `SCAN` , `Shortest Seek Time First`).

LOOK Algorithm: The elevator continues in the current direction as long as there are pending requests ahead; it reverses direction only when no further requests exist in that direction.

```

+-----+
| Elevator |
+-----+
      |
+-----+
| <<Strategy>> |

```



Implementation

Java



```

package com.design.elevator;

import java.util.TreeSet;

public class ElevatorSystem {

    public enum Direction { UP, DOWN, IDLE }

    public static class Elevator {
        private final int id;
        private int currentFloor = 0;
        private Direction direction = Direction.IDLE;

        // Up requests sorted ascending; down requests sorted descending
        private final TreeSet<Integer> upRequests = new TreeSet<>();
        private final TreeSet<Integer> downRequests = new TreeSet<>((a, b) -> b - a);

        public Elevator(int id) { this.id = id; }
    }
}

```

```
public synchronized void addRequest(int targetFloor) {
    if (targetFloor == currentFloor) return;
    if (targetFloor > currentFloor) {
        upRequests.add(targetFloor);
    } else {
        downRequests.add(targetFloor);
    }
    if (direction == Direction.IDLE) {
        direction = (targetFloor > currentFloor) ? Direction.UP : Direction.DOWN;
    }
}

public synchronized void step() {
    if (direction == Direction.UP) {
        if (!upRequests.isEmpty()) {
            currentFloor = upRequests.pollFirst();
        } else if (!downRequests.isEmpty()) {
            direction = Direction.DOWN;
            currentFloor = downRequests.pollFirst();
        } else {
            direction = Direction.IDLE;
        }
    } else if (direction == Direction.DOWN) {
        if (!downRequests.isEmpty()) {
            currentFloor = downRequests.pollFirst();
        } else if (!upRequests.isEmpty()) {
            direction = Direction.UP;
            currentFloor = upRequests.pollFirst();
        } else {
            direction = Direction.IDLE;
        }
    }
}

public int getCurrentFloor() { return currentFloor; }
```

```

    public Direction getDirection() { return direction; }
    public int getId() { return id; }
}

public interface DispatchStrategy {
    Elevator selectElevator(java.util.List<Elevator> elevators, int floor, Direction direction);
}

public static class NearestElevatorStrategy implements DispatchStrategy {
    @Override
    public Elevator selectElevator(java.util.List<Elevator> elevators, int floor, Direction direction) {
        Elevator best = null;
        int minDistance = Integer.MAX_VALUE;

        for (Elevator e : elevators) {
            int distance = Math.abs(e.getCurrentFloor() - floor);
            // Favor idle elevators or those already moving toward the destination
            if (e.getDirection() == Direction.IDLE || e.getDirection() == direction) {
                if (distance < minDistance) {
                    minDistance = distance;
                    best = e;
                }
            }
        }
        return (best != null) ? best : elevators.get(0);
    }
}

```

System 12: Parking Lot Management System

Design Patterns Applied

Factory Pattern: Dynamically allocates `ParkingSpot` subclasses (`Compact` , `Large` , `Motorbike`).

Strategy Pattern: Implements dynamic fee calculation based on duration and vehicle classification (`HourlyFeeCalculationStrategy`).

Implementation

Java



```
package com.design.parkinglot;

import java.time.Duration;
import java.time.Instant;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class ParkingLotSystem {

    public enum VehicleType { MOTORBIKE, CAR, TRUCK }

    public abstract static class Vehicle {
        private final String licensePlate;
        private final VehicleType type;

        public Vehicle(String licensePlate, VehicleType type) {
            this.licensePlate = licensePlate;
            this.type = type;
        }
        public VehicleType getType() { return type; }
        public String getLicensePlate() { return licensePlate; }
    }
}
```

```
public static class Car extends Vehicle {
    public Car(String licensePlate) { super(licensePlate, VehicleType.CAR); }
}

public abstract static class ParkingSpot {
    private final String spotId;
    private boolean occupied;
    private Vehicle currentVehicle;

    public ParkingSpot(String spotId) { this.spotId = spotId; }

    public abstract boolean canFitVehicle(Vehicle v);

    public synchronized void assignVehicle(Vehicle v) {
        if (!canFitVehicle(v) || occupied) throw new IllegalStateException("Spot unavailable");
        this.currentVehicle = v;
        this.occupied = true;
    }

    public synchronized void vacate() {
        this.currentVehicle = null;
        this.occupied = false;
    }

    public boolean isOccupied() { return occupied; }
    public String getSpotId() { return spotId; }
}

public static class CompactSpot extends ParkingSpot {
    public CompactSpot(String spotId) { super(spotId); }
    @Override
    public boolean canFitVehicle(Vehicle v) {
        return v.getType() == VehicleType.CAR || v.getType() == VehicleType.MOTORBIKE;
    }
}
```

```
}
}
```

```
public record ParkingTicket(String ticketId, String spotId, String plate, Instant ent
```

```
public static class ParkingLot {
    private final List<ParkingSpot> spots = new ArrayList<>();
    private final Map<String, ParkingTicket> activeTickets = new ConcurrentHashMap<>()
```

```
public void addSpot(ParkingSpot spot) { spots.add(spot); }
```

```
public synchronized ParkingTicket parkVehicle(Vehicle v) {
    for (ParkingSpot spot : spots) {
        if (!spot.isOccupied() && spot.canFitVehicle(v)) {
            spot.assignVehicle(v);
            String ticketId = UUID.randomUUID().toString();
            ParkingTicket ticket = new ParkingTicket(ticketId, spot.getSpotId(),
            activeTickets.put(ticketId, ticket);
            return ticket;
        }
    }
    throw new RuntimeException("No available spot for vehicle: " + v.getType());
}
```

```
public synchronized double unParkVehicle(String ticketId, FeeStrategy feeStrategy) {
    ParkingTicket ticket = activeTickets.remove(ticketId);
    if (ticket == null) throw new IllegalArgumentException("Invalid ticket ID.");

    ParkingSpot assignedSpot = spots.stream()
        .filter(s -> s.getSpotId().equals(ticket.spotId()))
        .findFirst()
        .orElseThrow();

    assignedSpot.vacate();
    long hours = Duration.between(ticket.entryTime(), Instant.now()).toHours() +
    return feeStrategy.calculateFee(hours);
}
```

```
}  
}  
  
public interface FeeStrategy { double calculateFee(long hours); }  
  
public static class FlatRateFeeStrategy implements FeeStrategy {  
    @Override  
    public double calculateFee(long hours) { return hours * 20.0; }  
}  
}
```

System 13: Distributed Pub/Sub Messaging Engine

Requirements & Architecture

In-memory broker handling dynamic Topic registration, Subscription fanout, and worker thread-pool processing.

Observer Pattern: Decouples subscribers from topic publishers.

Thread pools handle message dispatch asynchronously, isolating slow consumers from publisher threads.

Implementation

Java



```
package com.design.pubsub;
```

```
import java.util.*;
```

```

import java.util.concurrent.*;

public class DistributedPubSubEngine {

    public record Message(String topic, String payload, long timestamp) {}

    public interface Subscriber {
        String getId();
        void onMessage(Message message);
    }

    public static class Topic {
        private final String name;
        private final Set<Subscriber> subscribers = ConcurrentHashMap.newKeySet();
        private final ExecutorService dispatcherPool = Executors.newCachedThreadPool();

        public Topic(String name) { this.name = name; }

        public void addSubscriber(Subscriber sub) { subscribers.add(sub); }
        public void removeSubscriber(Subscriber sub) { subscribers.remove(sub); }

        public void publish(Message message) {
            for (Subscriber sub : subscribers) {
                dispatcherPool.submit(() -> {
                    try {
                        sub.onMessage(message);
                    } catch (Exception ex) {
                        System.err.printf("Error dispatching to %s: %s\n", sub.getId(), ex.getMessage());
                    }
                });
            }
        }
    }

    public static class PubSubBroker {

```

```

private final Map<String, Topic> topics = new ConcurrentHashMap<>();

public void createTopic(String topicName) {
    topics.putIfAbsent(topicName, new Topic(topicName));
}

public void subscribe(String topicName, Subscriber subscriber) {
    Topic topic = topics.get(topicName);
    if (topic == null) throw new IllegalArgumentException("Topic does not exist:");
    topic.addSubscriber(subscriber);
}

public void publish(String topicName, String payload) {
    Topic topic = topics.get(topicName);
    if (topic == null) throw new IllegalArgumentException("Topic does not exist:");
    topic.publish(new Message(topicName, payload, System.currentTimeMillis()));
}
}
}
}

```

System 14: Snake and Ladder Multiplayer Game

Object-Oriented Domain Entities

Board: Encapsulates cells containing Jump conditions (Snake , Ladder).

Player: Holds positional state, ID, and turn order.

Dice: Strategy abstraction for rolling (supports 1–6 dice or rigged testing dice).

Implementation

Java



```
package com.design.snakeladder;

import java.util.*;

public class SnakeAndLadderGame {

    public record Jump(int start, int end) {
        public Jump {
            if (start == end) throw new IllegalArgumentException("Jump cannot start and end at the same position");
        }
    }

    public static class Board {
        private final int size;
        private final Map<Integer, Integer> jumps = new HashMap<>();

        public Board(int size) { this.size = size; }

        public void addSnake(int head, int tail) {
            if (head <= tail) throw new IllegalArgumentException("Snake head must be higher than tail");
            jumps.put(head, tail);
        }

        public void addLadder(int bottom, int top) {
            if (bottom >= top) throw new IllegalArgumentException("Ladder bottom must be higher than top");
            jumps.put(bottom, top);
        }

        public int resolvePosition(int currentPos) {
            return jumps.getOrDefault(currentPos, currentPos);
        }
    }
}
```

```
        public int getSize() { return size; }
    }

    public static class Player {
        private final String name;
        private int position = 0;

        public Player(String name) { this.name = name; }
        public String getName() { return name; }
        public int getPosition() { return position; }
        public void setPosition(int position) { this.position = position; }
    }

    public static class Dice {
        private final int sides;
        private final Random random = new Random();

        public Dice(int sides) { this.sides = sides; }
        public int roll() { return random.nextInt(sides) + 1; }
    }

    public static class GameController {
        private final Board board;
        private final Dice dice;
        private final Deque<Player> players = new ArrayDeque<>();
        private boolean isFinished = false;

        public GameController(Board board, Dice dice, List<Player> playerList) {
            this.board = board;
            this.dice = dice;
            this.players.addAll(playerList);
        }

        public void playTurn() {
```

```

        if (isFinished) return;

        Player currentPlayer = players.pollFirst();
        int roll = dice.roll();
        int target = currentPlayer.getPosition() + roll;

        if (target <= board.getSize()) {
            target = board.resolvePosition(target);
            currentPlayer.setPosition(target);
        }

        if (currentPlayer.getPosition() == board.getSize()) {
            System.out.println("Player " + currentPlayer.getName() + " has won the game");
            isFinished = true;
            return;
        }

        players.addLast(currentPlayer);
    }

    public boolean isFinished() { return isFinished; }
}

```

System 15: Online Movie Ticket Booking System (BookMyShow)

Concurrency & Race Condition Strategy

Multiple customers selecting the same seat concurrently leads to race conditions.

Two-Phase Reservation Pattern:

1. *Temporary Lock:* When a customer clicks a seat, the seat transitions from `AVAILABLE` to `LOCKED` with an expiration timestamp (10 minutes) inside Redis or using an optimistic locking version check in PostgreSQL.
2. *Permanent Reservation:* Transitions to `BOOKED` only upon verified payment confirmation. If the payment gateway drops or the timer expires, the lock releases back to `AVAILABLE`.

Implementation

Java



```
package com.design.bookmyshow;

import java.time.Instant;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class TicketBookingSystem {

    public enum SeatStatus { AVAILABLE, LOCKED, BOOKED }

    public static class Seat {
        private final String seatId;
        private SeatStatus status = SeatStatus.AVAILABLE;
        private Instant lockExpirationTime = Instant.MIN;

        public Seat(String seatId) { this.seatId = seatId; }

        public synchronized boolean lockSeat(int holdDurationSeconds) {
            if (status == SeatStatus.BOOKED) return false;
```

```
// If already locked, verify whether lock has expired
if (status == SeatStatus.LOCKED && Instant.now().isBefore(lockExpirationTime)
    return false;
}

this.status = SeatStatus.LOCKED;
this.lockExpirationTime = Instant.now().plusSeconds(holdDurationSeconds);
return true;
}

public synchronized boolean confirmBooking() {
    if (status == SeatStatus.LOCKED && Instant.now().isBefore(lockExpirationTime)
        this.status = SeatStatus.BOOKED;
        return true;
    }
    return false;
}

public synchronized void releaseLock() {
    if (status != SeatStatus.BOOKED) {
        this.status = SeatStatus.AVAILABLE;
        this.lockExpirationTime = Instant.MIN;
    }
}

public SeatStatus getStatus() { return status; }
public String getSeatId() { return seatId; }
}

public static class ShowService {
    private final Map<String, Seat> seatMap = new ConcurrentHashMap<>();

    public void addSeat(String seatId) {
        seatMap.put(seatId, new Seat(seatId));
    }
}
```

```

public boolean holdSeats(List<String> seatIds, int ttlSeconds) {
    List<Seat> successfullyLocked = new ArrayList<>();
    for (String id : seatIds) {
        Seat seat = seatMap.get(id);
        if (seat != null && seat.lockSeat(ttlSeconds)) {
            successfullyLocked.add(seat);
        } else {
            // Rollback previously acquired locks if any seat in the selection failed
            successfullyLocked.forEach(Seat::releaseLock);
            return false;
        }
    }
    return true;
}

public boolean finalizeBooking(List<String> seatIds) {
    for (String id : seatIds) {
        Seat seat = seatMap.get(id);
        if (seat == null || !seat.confirmBooking()) {
            return false;
        }
    }
    return true;
}
}
}
}

```

5. Architectural Synthesis Matrix: Design Trade-offs & Production Decision Framework

Problem Archetype	Primary Latency Bottleneck	Storage Architecture	Concurrency Strategy	High-Availability Tac
Media Streaming (Netflix/YouTube)	Massive Outbound Bandwidth	S3 Object Store + Regional ISP OpenConnect CDNs	Async Queue-driven chunk transcoding	Anycast Geo-DNS failover
Geospatial Dispatch (Uber)	High Ingestion QPS	Redis Geospatial / In- memory H3 spatial indexing	Spatial partitioning via Uber H3 index	Kafka replica replication
Transactional Orders (Food Delivery)	Database Lock Contention	MySQL InnoDB + PostgreSQL JSONB Schemas	Distributed Redis token decrement + Saga orchestration	Read-write replicas with Multi-AZ failove
Real-time Chat (WhatsApp)	Open TCP Stream Maintenance	ScyllaDB/Cassandra (Unsent queues only)	In-memory connection socket maps	Ephemeral message drop upon client AC
Timeline Feed (Twitter/X)	Fanout Amplification	Redis Lists + Relational / TiDB clusters	Hybrid: Push for regular, Pull for celebrities	Precomputed cache warmers
High-Volume Crawl (Google)	Network I/O & DNS Latency	Distributed Object Storage + BigTable	Politeness queues with per-host delay locks	Dynamic IP rotation across fetcher pool:

6. End-to-End Self-Study Web Platform Architecture

To structure this material in*

i wanted a pdf and also these are not detailed and you not included every tools and tech i used , and also i wanted 15 example of fully solved for hld and lld take a time and give a fully prepared pdf . with examples . make sure to make that pdf fully explained.

[Next.js 14 Client Ap

(REST APIs / SSE Stream over TLS)
V
[FastAPI Microservice (Python 3.11 Backend)]
| | |

Java



```
package com.system.elevator;

import java.util.*;

public class ElevatorControlSystem {

    public enum Direction { UP, DOWN, IDLE }

    public static class ElevatorCar {
        private final int id;
        private int currentFloor = 0;
        private Direction direction = Direction.IDLE;

        // Ascending for upward trips, descending for downward trips
        private final TreeSet<Integer> upStops = new TreeSet<>();
        private final TreeSet<Integer> downStops = new TreeSet<>(Collections.reverseOrder());

        public ElevatorCar(int id) {
            this.id = id;
        }
    }
}
```

```

public synchronized void addDestination(int floor) {
    if (floor == currentFloor) return;
    if (floor > currentFloor) {
        upStops.add(floor);
        if (direction == Direction.IDLE) direction = Direction.UP;
    } else {
        downStops.add(floor);
        if (direction == Direction.IDLE

```

Module 1: Distributed Systems Core Foundations

1.1 Scaling Models: Vertical vs. Horizontal

```

* **Scale-Up (Vertical Scaling):** Increasing single-host resources (CPU clock/cores, RAM)
  * **Strengths:** Zero distributed coordination overhead, consistent memory access, no network latency
  * **Trade-offs:** Hard hardware boundaries, non-linear hardware cost increases, unavoidable capacity limits
* **Scale-Out (Horizontal Scaling):** Distributing compute and storage across independent nodes
  * **Strengths:** Practically boundless horizontal headroom, fault isolation, elastic provisioning
  * **Trade-offs:** Distributed state management, network serialization, consistency models

```

1.2 Network Transport & IPC Primitives

COMMUNICATION MODELS OVERVIEW			
Short Polling	Client -> Server	Periodic HTTP round-trips	
Long Polling	Client <-> Server	Server holds HTTP request pending	
WebSockets	Client <==> Server	Persistent bidirectional TCP stream	

| Server-Sent Events | Client <--- Server | Unidirectional streaming over HTTP |

+-----+-----+-----+-----+

Short Polling vs. Long Polling vs. WebSockets vs. SSE

1. **Short Polling:** Repeated HTTP requests at an interval Δt . Inefficient; leads to high latency.
2. **Long Polling:** The server suspends response completion until an event occurs or a timeout.
3. **WebSockets (RFC 6455):** An initial HTTP handshake negotiates a bidirectional, frame-based connection.
4. **Server-Sent Events (SSE):** Long-lived HTTP connection using 'text/event-stream'. No client-to-server messages.

REST vs. gRPC

- * **REST (Representational State Transfer):** Typically operates over HTTP/1.1 or HTTP/2.
- * **gRPC:** Strictly HTTP/2 transport utilizing binary serialization (Protocol Buffers).
- * **Features:** Multiplexed streams, bidirectional channels, zero-copy parsing, strongly typed.

1.3 Traffic Ingress, Proxies & Load Balancing

◀ 

```

Internet
|
[ Layer 4 LB ] (TCP/UDP: IP + Port routing, e.g., AWS NLB / HAProxy)
|

```

[Layer 7 LB] (HTTP: TLS Offload, Path-based routing, e.g., Envoy / NGINX)
/ \

[/orders] [/users]
| |
[Order Svc] [User Svc]

Layer 4 vs. Layer 7 Routing

- * **Layer 4 (L4):** Operates at the transport layer (TCP/UDP). Inspects packet headers wi
- * **Layer 7 (L7):** Operates at the application layer. Parses HTTP requests, offloads TLS

Dispatching Algorithms

- * **Round-Robin / Weighted Round-Robin:** Predictable; assumes homogeneous task runtimes
- * **Least Connections / Weighted Least Connections:** Routes traffic to the host servicir
- * **IP / Consistent Hash Ring:** Preserves spatial affinity for caching layers and statef

1.4 Consistent Hashing Mechanics



Node A (θ°)
/ \

Key 3 / \ Key 1
 / \

Node C (240°) ----- Node B (120°)

Key 2

In basic modular partitioning ($\text{node} = \text{hash}(\text{key}) \bmod N$), altering

The Hash Ring & Virtual Nodes (Vnodes)

- * Nodes and data keys map to an integer domain $[0, 2^{32}-1]$ along a logical ring.
- * Keys are mapped to the first node located clockwise from the key's hash value.
- * Node addition/removal limits key remapping to $\frac{K}{N}$ on average.
- * **Virtual Nodes (Vnodes):** Each physical machine receives V distinct virtual tokens $\propto \frac{\text{Target Tokens per Host}}{N}$ in $[100, 256]$
- * **Why:** Prevents load skew on non-uniform token placement, and balances redistribution

1.5 Distributed Caching & Invalidation Architecture



Cache-Aside (Lazy)	Read-Through	Write-Through	Write-Behind (Write-Back)
App -> Cache	App -> Cache	App -> Cache	App -> Cache
App -> DB (on miss)	Cache -> DB (on miss)	Cache -> DB (Sync)	Cache -> Async Queue -> DB
App -> Cache (re-populate)			

Invalidation Strategies

- * **Cache-Aside (Lazy Population):** Read misses pull from primary storage and backfill cache
- * **Read-Through / Write-Through:** Cache sits directly in the read/write path. Write-through updates cache and disk
- * **Write-Behind (Write-Back):** Writes update cache instantly; disk updates queue asynchronously

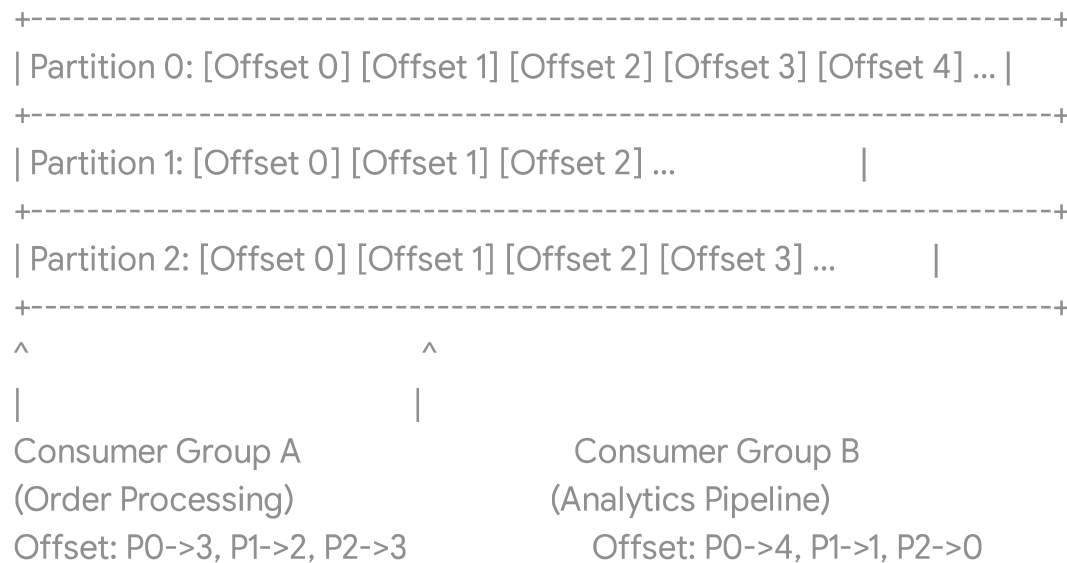
Cache Eviction Policies

- * **Least Recently Used (LRU):** Removes elements whose last access is furthest in the past
- * **Least Frequently Used (LFU):** Drops items with the lowest access frequencies. Uses frequency counter
- * **Time-to-Live (TTL):** Enforces passive removal on reads alongside an active probabilistic eviction

1.6 Distributed Messaging: Apache Kafka Deep Dive



TOPIC: "order-events"



Topics, Partitions, and Offsets

- * A **Topic** is a logical event category partitioned across disk storage systems.
- * A **Partition** is an ordered, immutable, append-only log sequence. Messages receive a
- * Partitioning enables horizontal read/write scale. Strict sequence ordering is guaranteed

Producers & Partition Keys

- * When a message includes a key:
 - $$\text{partition} = \text{MurmurHash2}(\text{key}) \bmod \text{num_partitions}$$

This routes messages with matching keys (such as 'account_id' or 'order_id') to the same partition.
- * If the key is null, messages distribute via round-robin or sticky partitioning.

Brokers, Leaders, and In-Sync Replicas (ISR)

- * Each partition has one **Leader** broker handling all read/write traffic, and $R-1$ **Follower** brokers.
- * **ISR (In-Sync Replicas)**: Replicas that actively fetch records within the 'replica.lag' threshold.
- * 'acks=all' (or 'acks=-1'): The leader defers write acknowledgments until all ISR members have acknowledged.

Consumer Groups & Scale Boundaries

- * Partitions within a topic are assigned to at most **one** consumer instance per Consumer Group.
- * If 'consumers > partitions', surplus consumers sit idle.
- * Consumers maintain independent commit pointers, allowing multiple downstream applications to consume from the same topic.

1.7 Data Consistency Models & Distributed Storage

The CAP & PACELC Theorems

- * **CAP Theorem**: Under a network partition (\$P\$), a distributed datastore must choose between Consistency (\$C\$) and Availability (\$A\$).
 - * **Consistency (\$C\$)**: All alive nodes return the identical, most recent write, or throw an error.
 - * **Availability (\$A\$)**: All operational nodes process reads/writes without guaranteeing consistency.
- * **PACELC Formulation**: If Partitioned (\$P\$), balance Availability (\$A\$) vs Consistency (\$C\$) with a latency factor (\$L\$).
 - * **Example**: DynamoDB/Cassandra choose \$PA/EL\$; relational databases with synchronous replication choose \$PC\$.

Relational Storage Engines vs. NoSQL

* **B+Tree Storage Engines (PostgreSQL / MySQL InnoDB):**

- * Optimizes read queries ($O(\log N)$) through balanced, high fan-out trees matching on keys
- * Writes require synchronous WAL (Write-Ahead Log) persistence and random-access page lookups

* **LSM-Tree Engines (Cassandra / RocksDB / ScyllaDB):**

- * Converts writes to sequential append-only disk operations via in-memory MemTables backed by memory
- * MemTables flush to immutable SSTables (Sorted String Tables).
- * Reads require checking Bloom Filters, MemTables, and multiple SSTable levels, shifting the read/write trade-off

1.8 Distributed Coordination & Cross-Service Transactions

Two-Phase Commit (2PC)

1. **Prepare Phase:** The coordinator dispatches a prepare statement. Resource participants respond with 'YES' or 'NO'.
2. **Commit Phase:** If all participants vote 'AGREE', the coordinator logs commit status and sends 'COMMIT' to all participants.
3. **Bottlenecks:** Blocking protocol. If the coordinator crashes mid-process, participants must wait for a timeout or recovery.

The Saga Pattern

Decomposes distributed transactions into chained local database transactions.

[Order Svc] (Create Order)

|

Kafka: "OrderCreated"

v

[Payment Svc] (Process Charge)

|

Kafka: "PaymentFailed" =====+

v

| (Compensating Rollback)

[Inventory Svc] (Skip) v
 [Order Svc] (Set Status: CANCELLED)

* ****Choreography:**** Microservices publish and listen to domain events across Kafka topics
 * ****Orchestration:**** A dedicated orchestrator state machine directs participants via sync
 * ****Compensating Transactions:**** Forward transactions must define backward-compensating a

Module 2: The Enterprise Backend & AI Integration Stack

2.1 Spring Boot Internal Architecture & Lifecycle



```
[ HTTP Request ]
|
[ Filter Chain: CorsFilter -> CsrfFilter -> JwtAuthenticationFilter ]
|
[ DispatcherServlet ]
|
[ HandlerMapping ] ---> Resolves Route Target
|
[ HandlerAdapter ] ---> Executes Target Method
|
[ Controller Advice / MessageConverter (Jackson) ]
|
[ JSON Response ]
```

DispatcherServlet Ingress Flow

- * Requests hit the servlet container and enter the Spring container through the front-controller.
- * 'HandlerMapping' matches incoming URLs to candidate '@RestController' routes.
- * 'HandlerAdapter' executes the controller action, leveraging 'HttpMessageConverters' (e.g., 'Jackson2HttpMessageConverter').
- * Method interception annotations ('@Transactional', '@PreAuthorize') execute dynamically.

Spring Security Context & JWT Verification Filter

- * **Filter Sequence:** The 'SecurityFilterChain' intercepts requests before reaching the application.
- * **State Lifecycle:**
 1. Incoming HTTP requests deliver a bearer token: 'Authorization: Bearer <token>'.
 2. 'JwtAuthenticationFilter' intercepts the request, runs signature verification (HMAC-SHA256).
 3. A validated token populates an 'Authentication' instance (e.g., 'UsernamePasswordAuthenticationToken').

```

```java
SecurityContextHolder.getContext().setAuthentication(auth);
```

```

 4. The downstream controller executes within the caller's authorized context.
 5. The thread-local context clears on request completion, maintaining stateless execution.

2.2 Relational Data Systems & JSONB Internals

B+Tree Indexing & Multi-Version Concurrency Control (MVCC)

- * Relational database tables organize pages inside B+Trees. Internal nodes contain routing pointers to leaf pages.
- * **MVCC:** Writes do not block reads, and reads do not block writes. When a row updates, a new version is created.

PostgreSQL JSONB Decomposition vs. Raw Text JSON

- * Raw 'JSON' stores text verbatim, requiring repetitive parsing on queries.
 - * **JSONB Internals:** Decomposes JSON keys and values into indexed binary structures. It uses a compact binary format.
 - * **GIN (Generalized Inverted Index):** Builds index maps across all interior paths, keys, and values.
- ```

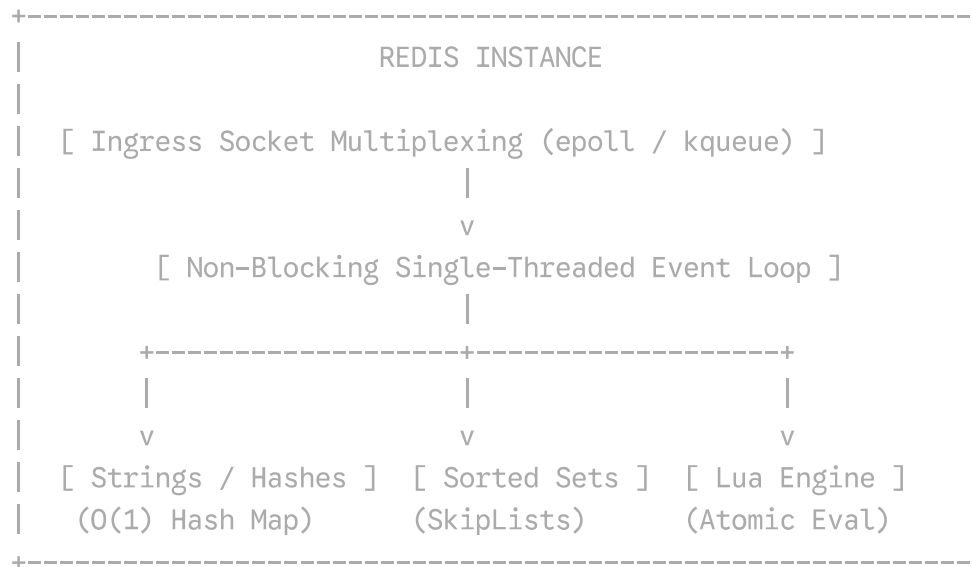
```sql
-- Create GIN index on jsonb attributes
CREATE INDEX idx_jsonb_gin ON table_name USING GIN (jsonb_column);
```

```

```
CREATE INDEX idx_listings_dynamic_attrs ON listings USING gin (dynamic_attributes);

-- Fast containment query leveraging the index
SELECT * FROM listings
WHERE dynamic_attributes @> '{"category": "vehicle", "transmission": "automatic"}';
```

## 2.3 Redis Internals & Concurrency Control



### Single-Threaded Event Loop & Data Structures

Redis runs on an event-driven, single-threaded execution loop using non-blocking I/O multiplexing ( `epoll` on Linux, `kqueue` on BSD/macOS ).

Commands run sequentially in-memory without multi-threaded locking overhead or CPU context switches.

Core data types map to optimized low-level data structures:

*Strings*: Simple Dynamic Strings (SDS) with pre-allocated buffers.

*Hashes*: ZipLists (for small payloads) migrating to Hash Tables.

*Sorted Sets (ZSETs)*: Dual-structure combining a Hash Map (for  $O(1)$  score lookups) and a SkipList (for  $O(\log N)$  insertion and range queries).

## Distributed Locking with TTL Safety

Basic `SETNX` commands without expirations risk perpetual deadlocks if the holding process crashes.

Bash



```
Atomic acquisition with auto-expiration
```

```
SET resource:lock:order_12345 "unique-uuid-token" NX PX 30000
```

`NX` : Set if Not Exists. Ensures only one caller captures the lock.

`PX 30000` : Applies a 30,000 ms TTL expiration limit.

**Safe Lock Release via Lua Script:** To avoid race conditions where process A clears process B's expired lock, release operations execute atomically in Lua:

Lua

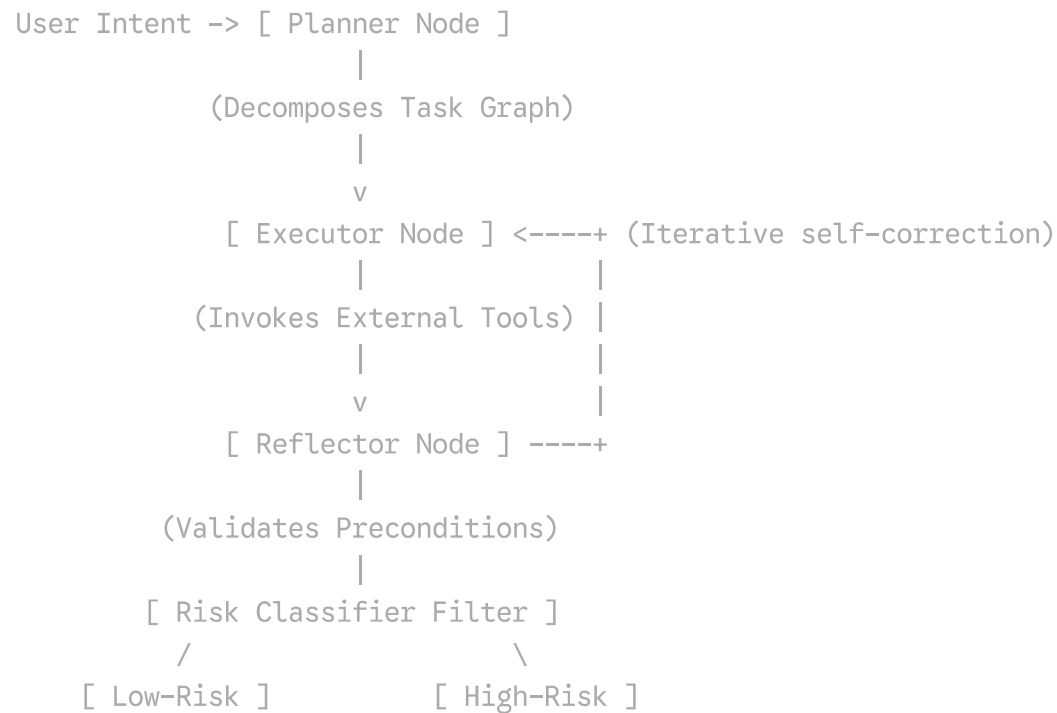


```

if redis.call("get", KEYS[1]) == ARGV[1] then
 return redis.call("del", KEYS[1])
else
 return 0
end

```

## 2.4 Agentic LLM Architectures: Planner-Executor-Reflector



|  
(Auto-Commit)      [ Human Approval Gate ]

## Component Roles

**Planner:** Evaluates the user query, inspects available tools and schemas, and plans an execution DAG (Directed Acyclic Graph).

**Executor:** Traverses the task DAG, invoking domain services via structured JSON function calling.

**Reflector:** Evaluates tool responses against expected schemas and business requirements. If execution errors or malformed payloads emerge, the reflector appends the error trace to context and prompts the planner to re-route.

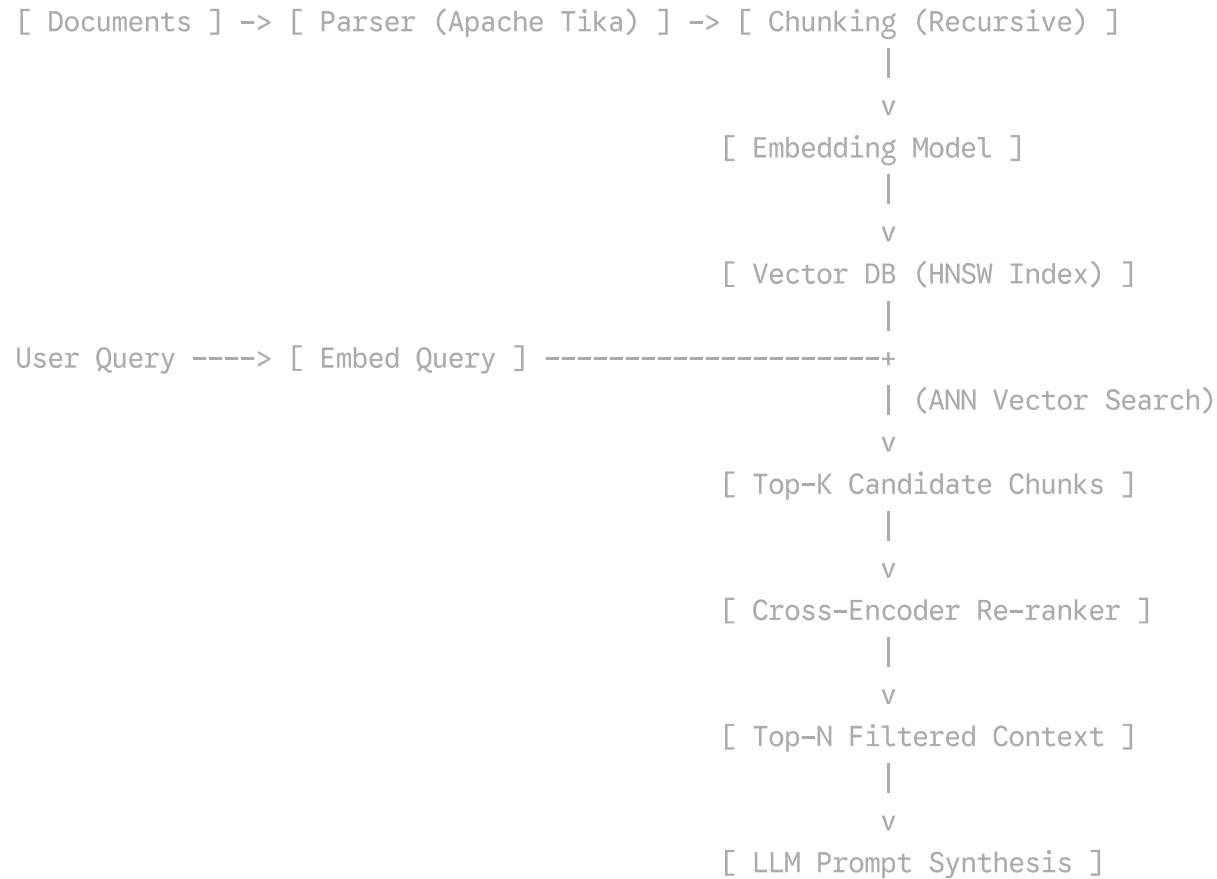
## Risk-Tiered Human-in-the-Loop (HITL) Integration

**Tier 1 (Zero / Low-Risk):** Read-only data queries and non-state-changing tasks execute automatically.

**Tier 2 (Medium-Risk):** Reversible writes (e.g., updating user notification settings) run automatically with audit logs.

**Tier 3 (High-Risk):** Irreversible or financial actions (e.g., updating bank account numbers, processing refunds > \$100). The workflow pauses, persists execution state to a database, and generates an administrative review event. It resumes only upon receiving an authorized signature.

## 2.5 Retrieval-Augmented Generation (RAG) Architecture



## Ingestion Pipeline

1. **Extraction:** Documents (PDFs, docs, images) pass through extractors like Apache Tika to isolate text and metadata.
2. **Chunking Strategies:** Text splits via recursive splitters into 500–1000 token windows with a 15–20% sliding overlap, preserving contextual meaning across split boundaries.

3. **Dense Embeddings:** Vector models project text chunks into high-dimensional space (e.g., 1536-dimensional embeddings).
4. **Vector Indexing (HNSW):** Hierarchical Navigable Small World graphs support Approximate Nearest Neighbor (ANN) search with logarithmic query times, trading small amounts of recall accuracy for high throughput.

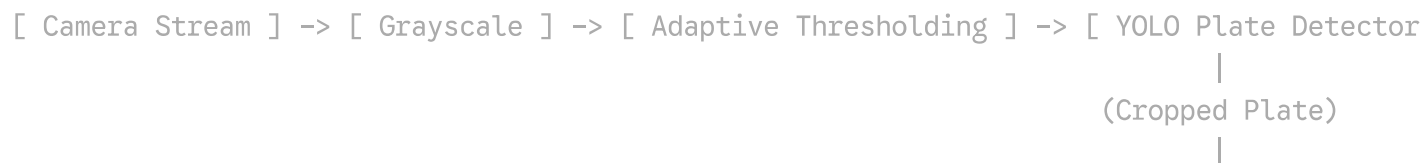
## Query Pipeline

1. The user's search text embeds using the same vector model.
2. ANN search retrieves the top- $K$  candidates based on cosine similarity or dot-product metrics:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$

3. **Cross-Encoder Re-ranking:** A cross-encoder reviews the top- $K$  chunks alongside the user query, scoring chunk relevance to pick the top- $N$  sources ( $N < K$ ).
4. **Context Injection:** The prompt context injects the top- $N$  text chunks, instructing the model to answer using only the provided reference context.

## 2.6 Computer Vision: Edge ANPR Pipeline



```
[Output Result] <-- [Regex Rules] <-- [CRNN / OCR Engine] <-- [Contrast Normalizat
```

## YOLO Detection Mechanics

YOLO (You Only Look Once) frames plate localization as an end-to-end regression problem.

Single-stage forward passes evaluate the input frame, predicting bounding box anchors, objectness scores, and class vectors across grid regions without candidate region proposals. This allows the model to process live video feeds at 30–60 FPS.

## Preprocessing Operations (OpenCV)

1. **Grayscale Conversion:** Reduces three-channel RGB inputs to single-channel intensity maps.
2. **Bilateral Filtering:** Smooths out noise while preserving sharp plate edge boundaries.
3. **Adaptive Thresholding:** Calculates dynamic thresholds across local neighborhoods, keeping text readable under direct headlights or shadow conditions.

## OCR (PaddleOCR / EasyOCR) & Parsing

1. Text detection layers (e.g., DBNet) generate bounding bounding polygons around character sequences.
2. Text recognition modules (CRNN with CTC loss or SVTR) convert pixel polygons into text strings.

- Post-processing regular expressions validate and clean the text against regional license plate formats (e.g., `^[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}$` ).

## Module 3: High-Level System Designs (HLD)

### System 1: Netflix (Global Video Streaming System)

#### Requirements

**Functional:** Upload and transcode videos; stream adaptive-bitrate video globally; save user playback positions.

**Non-Functional:** Low startup latency ( $< 500$  ms), 99.999% uptime, support for high read-to-write traffic ratios (1000 : 1).

#### Scale Estimations

Active users: 100 Million daily active users (DAU).

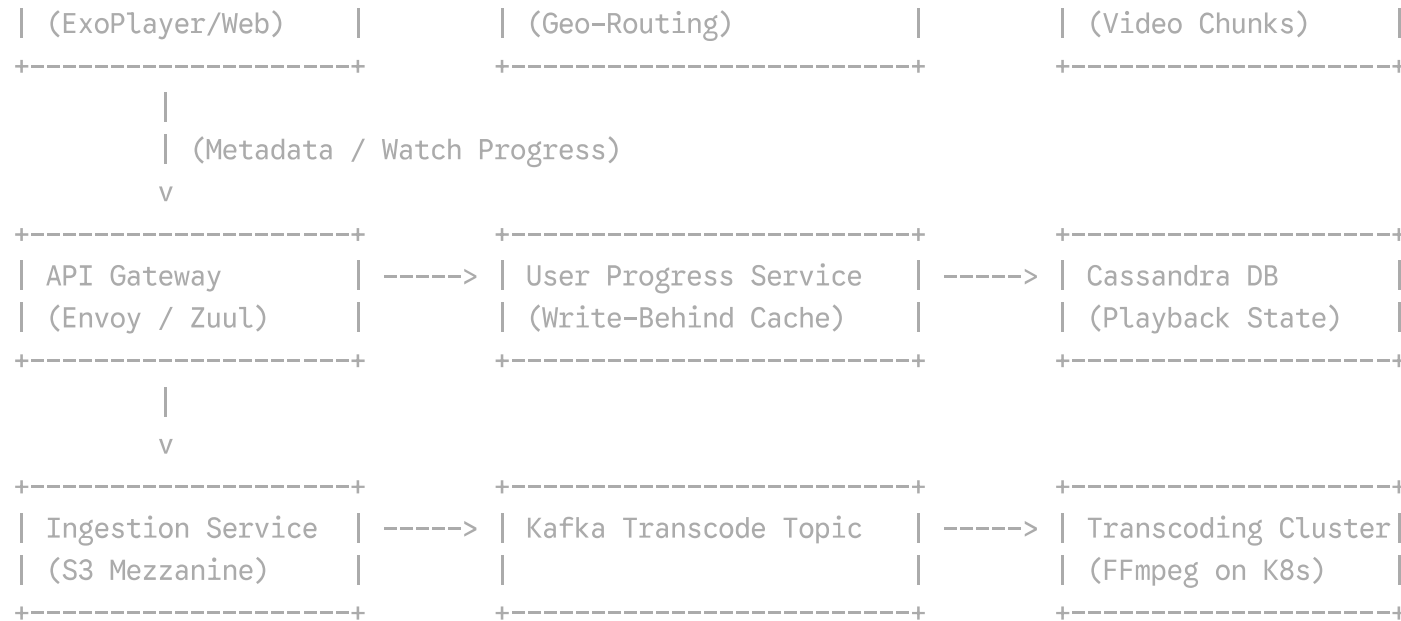
Peak concurrent streams: 15 Million.

Average video bitrate: 3 Mbps across profiles.

Outbound bandwidth:  $15 \text{ Million} \times 3 \text{ Mbps} = 45 \text{ Tbps}$  (distributed across Edge CDNs).

#### Architecture Diagram





## Transcoding & Edge Distribution

1. Video masters upload to temporary S3 mezzanine buckets.
2. S3 events publish to Kafka, triggering the transcoding engine.
3. Transcoding workers slice videos into independent 2- to 6-second segments, generating streams across multiple codecs (AV1, HEVC, H.264) and resolutions (360p to 4K).
4. Manifest files are generated: `.m3u8` for HLS (HTTP Live Streaming) and `.mpd` for DASH (Dynamic Adaptive Streaming over HTTP).
5. Slices push to regional Open Connect Appliances (custom FreeBSD storage boxes inside ISP data centers).

## Data Modeling: Cassandra

Cassandra handles playback state updates thanks to its high write throughput.

SQL



```
CREATE KEYSPACE netflix_streaming WITH replication = {
 'class': 'NetworkTopologyStrategy',
 'us-east': 3,
 'eu-west': 3
};
```

```
CREATE TABLE netflix_streaming.user_playback_state (
 user_id uuid,
 profile_id uuid,
 video_id uuid,
 last_playback_position_sec int,
 total_duration_sec int,
 completed_state boolean,
 updated_at timestamp,
 PRIMARY KEY ((user_id, profile_id), video_id)
) WITH CLUSTERING ORDER BY (video_id ASC);
```

*Why:* The compound partition key `(user_id, profile_id)` clusters watch histories onto a single partition, enabling fast queries for "Continue Watching" feeds.

## System 2: Uber / Grab (Geospatial Ride Dispatch Platform)

### Requirements

**Functional:** Ingest driver locations every 4 seconds; query nearby drivers; manage match requests; track active rides.

**Non-Functional:** Location ingestion latency < 500 ms; high availability under network partitions; consistency across ride reservation states.

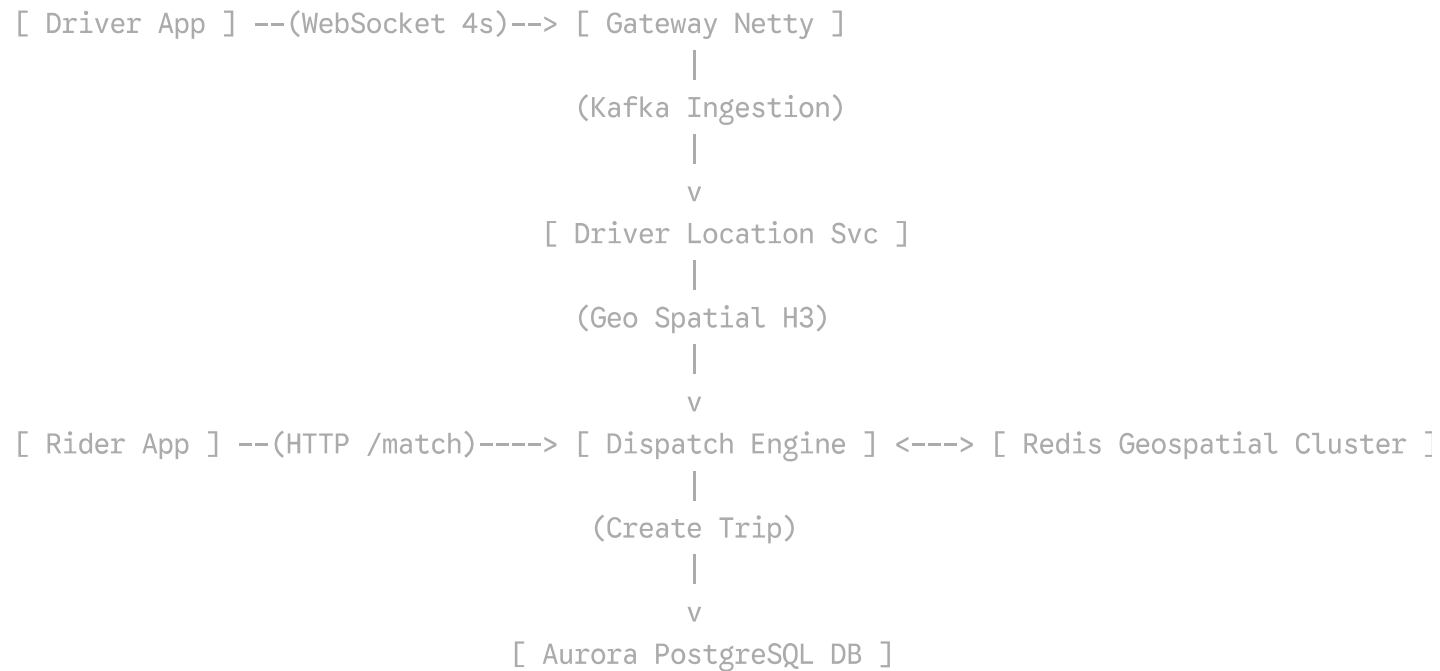
## Scale Estimations

Active Drivers: 1 Million active connections.

Location write load: 1 Million/4 sec = 250,000 QPS.

Active Riders: 10 Million active apps.

## Architecture Diagram



## Geospatial Partitioning: H3 Hexagonal Index

Uber's H3 index projects the globe onto a hexagonal grid.

Unlike rectangular or square grids (such as Geohash or S2), hexagons maintain uniform distances to all 6 adjacent neighbors. This simplifies radius searches, dynamic surge clustering, and dispatch routing.

Driver coordinates resolve to resolution level 8 (average hexagon area  $\approx 0.737 \text{ km}^2$ , edge radius  $\approx 461 \text{ m}$ ).

## Redis Geospatial Storage

Spatial points store in Redis Sorted Sets using 52-bit integer representations of coordinates:

Bash



```
Store driver location
```

```
GEOADD drivers:res8:8826856235ffffff 77.2090 28.6139 "driver_4091"
```

```
Search drivers within 3km
```

```
GEOSEARCH drivers:res8:8826856235ffffff FROMLONLAT 77.2090 28.6139 BYRADIUS 3 KM ASC
```



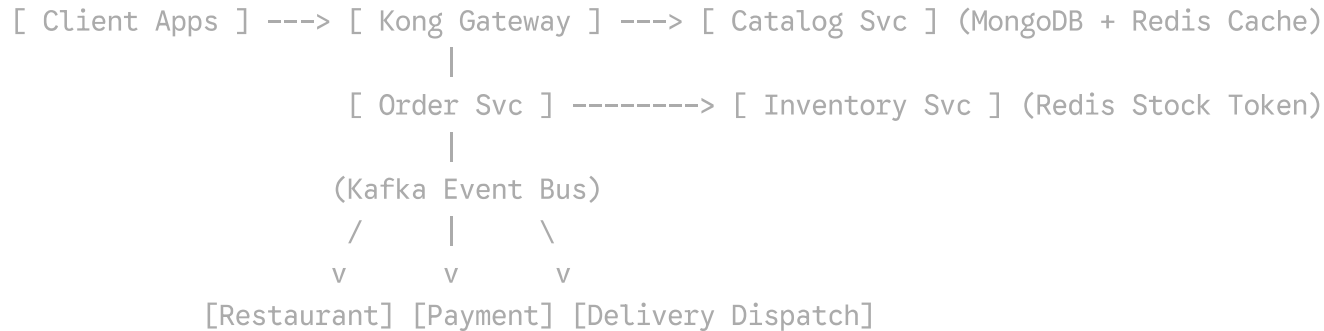
## System 3: Food Delivery Platform (DoorDash / Swiggy)

### Requirements

**Functional:** Search restaurant menus; manage carts; run checkout transactions; provide multi-party tracking (Customer, Restaurant, Courier).

**Non-Functional:** Prevent over-ordering on low stock; keep payment-to-order operations ACID-compliant; scale through peak meal periods.

## Architecture Diagram



## Inventory Race Conditions & Atomic Decrements

Traditional `SELECT ... FOR UPDATE` operations create database lock contention during high-demand flash sales. Instead, inventory pre-allocates using an atomic Lua script in Redis:

Lua



```

local stock_key = KEYS[1]
local requested_qty = tonumber(ARGV[1])
local current_stock = tonumber(redis.call('get', stock_key) or "0")

if current_stock >= requested_qty then
 redis.call('decrby', stock_key, requested_qty)
end

```

```
 return 1 -- Allocation Approved
else
 return 0 -- Insufficient Inventory
end
```

## The Saga Lifecycle Orchestration

1. Order Service registers an order in state `PENDING_PAYMENT` .
2. Payment Service charges the user's card.  
*If Payment Fails:* Order transitions to `PAYMENT_FAILED` and the process halts.
3. Restaurant Service receives the order.  
*If Rejected by Restaurant:* Order transitions to `REJECTED` ; a compensating refund transaction triggers on the Payment Service; the Redis inventory reservation reverts.
4. *If Accepted:* Order moves to `PREPARING` and the system dispatches a delivery courier.

## System 4: WhatsApp / Telegram (Secure Messaging System)

### Requirements

**Functional:** 1-to-1 chat; group conversations; delivery statuses (Sent, Delivered, Read); offline message storage.

**Non-Functional:** End-to-end encryption; sub-second delivery; zero plain-text storage on application servers.

### Architecture Diagram



## Connection Management

Clients maintain persistent, full-duplex TCP connections over mutual TLS using a binary protocol or WebSockets.

A distributed Redis cluster tracks active sessions:

Key: `session:user_id`  $\Rightarrow$  Value: `edge_gateway_pod_ip`

When User A sends a message to User B:

1. The Gateway reads the session mapping for User B.
2. If User B is online, the message forwards across internal networks directly to User B's gateway host, which delivers it over the open TCP socket.
3. If User B is offline, the encrypted payload appends to User B's mailbox in ScyllaDB, and an external push notification triggers (APNs/FCM).
4. Once User B reconnects, their client pulls unread payloads sequentially. The server removes records from temporary storage upon receiving delivery confirmations.

## Cryptographic Guarantees (The Signal Protocol)

**X3DH (Extended Triple Diffie-Hellman):** Performs initial key agreements and establishes shared secrets even if one party is offline.

**Double Ratchet Algorithm:** Derives new single-use symmetric encryption keys for every message sent using KDF (Key Derivation Function) chains. This provides Forward Secrecy (compromised keys cannot decrypt past messages) and Break-in Recovery (compromised keys cannot decrypt future communications).

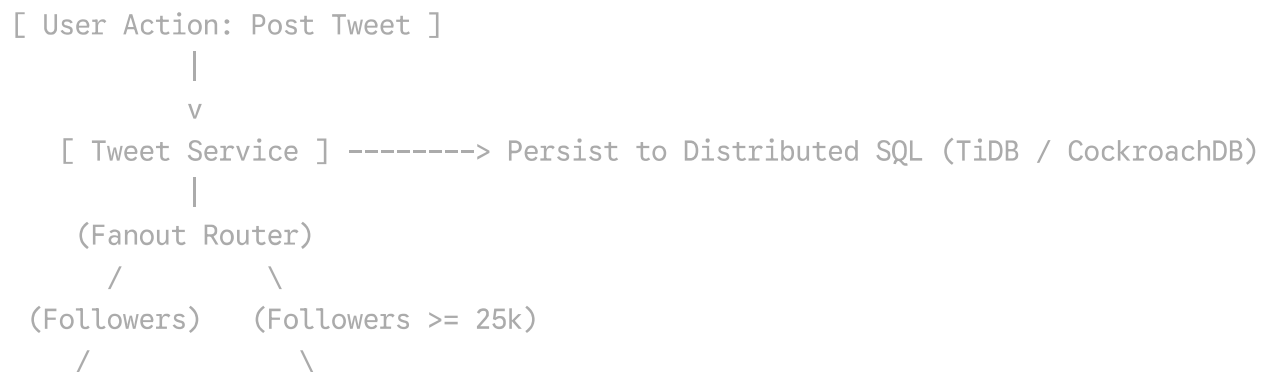
## System 5: Twitter / X (Timeline Engine & Social Graph)

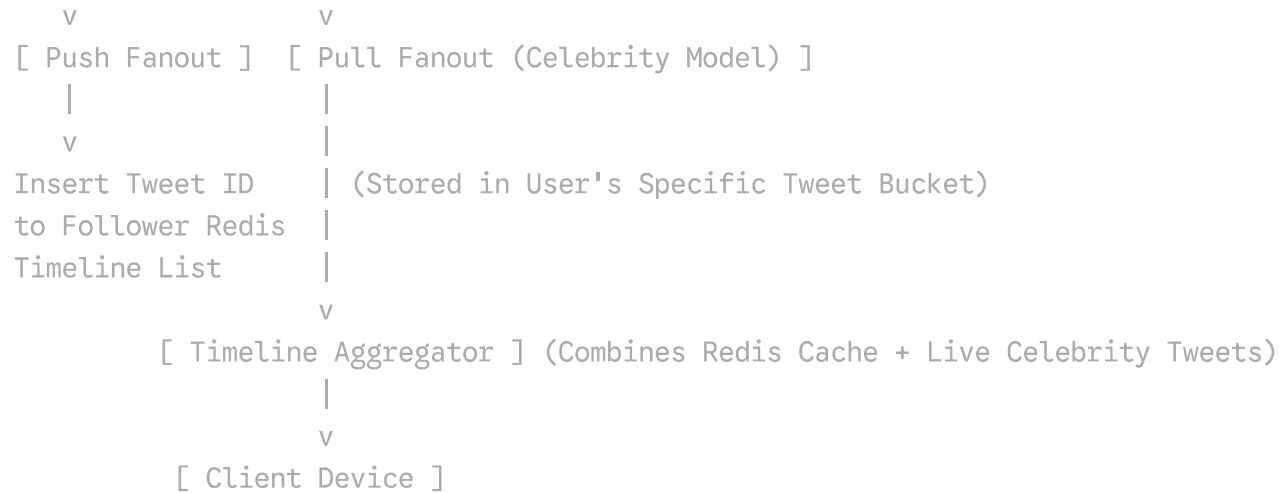
### Requirements

**Functional:** Post tweets; follow/unfollow users; view home timelines; view profile timelines.

**Non-Functional:** Read latency  $< 100$  ms; manage high read/write traffic ratios (100 : 1); scale through celebrity fanout spikes.

### Architecture Diagram





## Fanout-on-Write (Push) vs. Fanout-on-Read (Pull)

### Fanout-on-Write (Push):

When a user posts a tweet, background workers lookup their followers and insert the `tweet_id` into every follower's Redis timeline list:

Bash

```
LPUSH timeline:user:<follower_id> <tweet_id>
LTRIM timeline:user:<follower_id> 0 799
```



*Strengths:* Inexpensive reads ( $O(1)$ ) via `LRANGE` .

*Bottlenecks:* When an account with 80 Million followers tweets, the system must write millions of records, generating substantial processing lag and cache thrashing.

### Fanout-on-Read (Pull):

Tweets persist only to the author's local timeline.

When a user loads their feed, the system fetches all followed user accounts and merges their recent tweets in-memory using a multi-way merge-sort.

*Bottlenecks:* Significant CPU overhead and high read latency for users following thousands of accounts.

### Hybrid Model:

Users with  $< 25,000$  followers use **Push** fanout.

Accounts with  $\geq 25,000$  followers (celebrity tier) use **Pull** fanout.

The timeline aggregator reads the user's cached Redis list, fetches recent tweets for any followed celebrities, and merges the results in memory sorted by timestamp.

## System 6: YouTube / TikTok (Video Ingestion & Playback)

### Requirements

**Functional:** Resumable large-file uploads; automated transcoding pipelines; streaming playback; recommendation ranking.

**Non-Functional:** Prevent partial upload corruptions; keep video playback buffering  $< 1\%$ ; ensure high edge-cache availability.

### Architecture Diagram



## Resumable Upload Architecture

1. The client registers an upload: `POST /api/v1/videos/uploads/initiate`.
2. The server creates an upload session ID and returns signed multipart upload URLs pointing to S3.
3. The client uploads the file in 8MB chunks using HTTP `PUT` requests, tracking ETags for each part.
4. If a network interruption occurs, the client queries uploaded parts via `GET /uploads/parts?uploadId=...` and resumes uploading from the last missing

offset.

5. On completion, the server merges the parts into a single master asset.

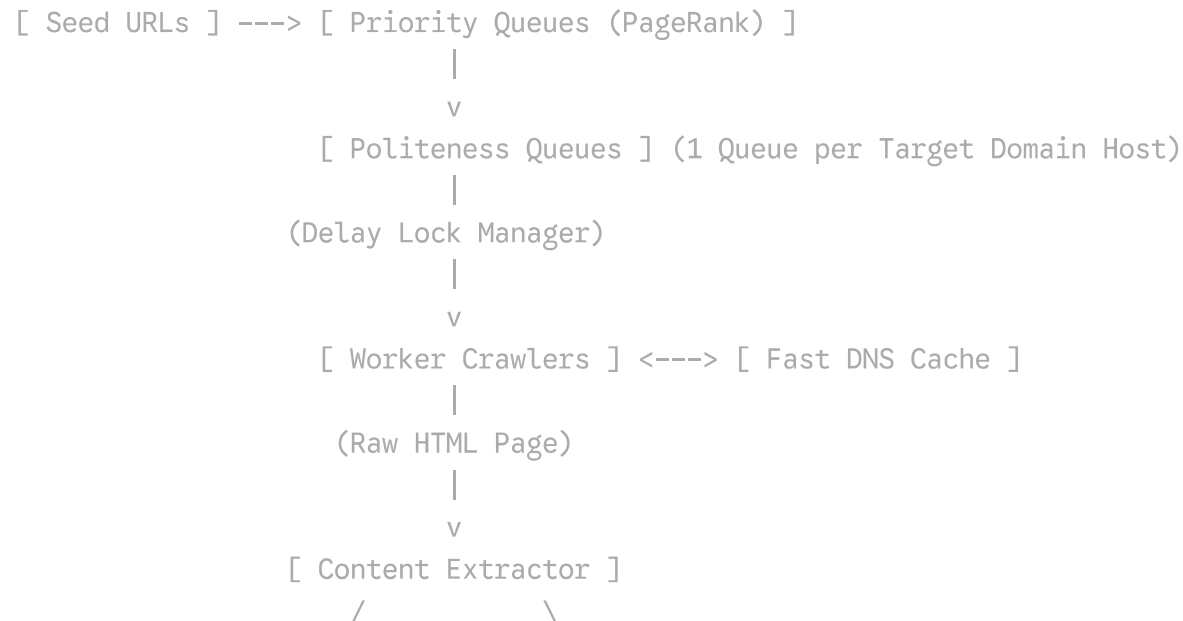
## System 7: Distributed Web Crawler (Search Engine Scale)

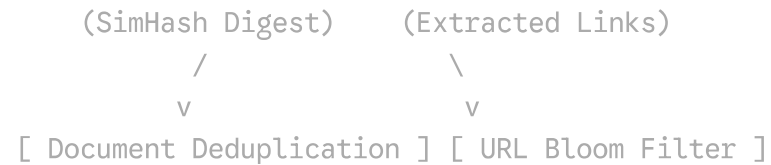
### Requirements

**Functional:** Crawl web pages starting from seed URLs; extract discovered links; deduplicate URLs and text; respect `robots.txt` .

**Non-Functional:** Politeness guarantees (avoid overloading target domains); crawl throughput  $\geq 50,000$  pages/sec; robust deduplication filtering.

### Architecture Diagram





## URL Frontier Architecture: Politeness & Prioritization

**Priority Queues (Front Queues):** Rank URLs by domain importance, PageRank scores, and update frequency.

**Politeness Queues (Back Queues):** Group URLs into distinct queues per target host (e.g., `queue:nytimes.com` ).

Each queue links to a timing lock:

$\text{Next Fetch Allowed} = \text{Previous Fetch Time} + \text{Host Delay (from robots.txt)}$

Worker threads dequeue URLs only after a host's timing lock expires, preventing accidental denial-of-service spikes against target websites.

## Deduplication Engine

**URL Filter:** A cluster-wide **Bloom Filter** confirms whether a URL was previously scheduled. Its bit-vector representation evaluates hashes in  $O(1)$  memory without querying relational disks.

**Content Similarity via SimHash:**

1. Tokenizes text content and calculates token frequency weights.
2. Hashes each token into a 64-bit fingerprint.

- 3. Combines bit positions: for bit position  $i$ , add weight if bit is 1, subtract if 0.
- 4. Yields a 64-bit summary fingerprint: if total sum  $> 0$ , set to 1; else set to 0.
- 5. Two pages are treated as duplicates if the Hamming distance between their SimHash values is  $\leq 3$ .

## System 8: Distributed Rate Limiter (API Defense Tier)

### Algorithm Trade-offs

| Algorithm       | Pros                          | Cons                         |
|-----------------|-------------------------------|------------------------------|
| Token Bucket    | Handles controlled bursts     | Requires dynamic state sync  |
| Leaky Bucket    | Enforces smooth outbound rate | Delays burst traffic         |
| Fixed Window    | Low memory footprint          | Traffic spikes at boundaries |
| Sliding Log     | High accuracy                 | Memory scales with traffic   |
| Sliding Counter | Low memory, high accuracy     | Approximates edge counts     |

### Production Implementation: Redis Sliding Window Log (Lua Script)

Lua



```
local rate_limit_key = KEYS[1]
local max_allowed = tonumber(ARGV[1])
local current_epoch = tonumber(ARGV[2])
```

```
local window_size = tonumber(ARGV[3])
local clear_boundary = current_epoch - window_size

-- 1. Evict entries older than the active window
redis.call('zremrangebyscore', rate_limit_key, 0, clear_boundary)

-- 2. Fetch the number of logged requests in the active window
local current_usage = redis.call('zcard', rate_limit_key)

if current_usage < max_allowed then
 -- 3. Log current request with a unique compound member
 local unique_token = current_epoch .. ":" .. redis.call('time')[2]
 redis.call('zadd', rate_limit_key, current_epoch, unique_token)
 redis.call('expire', rate_limit_key, window_size)
 return 1 -- Allowed
else
 return 0 -- Rate Limited (HTTP 429)
end
```

## Module 4: Low-Level System Designs (LLD) & Object-Oriented Implementations

### System 9: Distributed URL Shortener (TinyURL)

#### Design Decisions & Rationale

**Base62 Encoding:** Base64 contains reserved URL characters ( + , / ). Base62 uses only alphanumeric characters ( [a-zA-Z0-9] ), avoiding URL encoding issues.

**Deterministic ID Translation:** Hashing URLs introduces collision resolution overhead. Instead, we generate unique 64-bit integer IDs (via an ID generator or database

sequences) and convert them to Base62. A 7-character Base62 string yields  $62^7 \approx 3.52$  Trillion combinations.

## Complete Java Implementation

Java



```
package com.system.lld.urlshortener;

import java.time.Instant;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

public class TinyUrlEngine {

 private static final String ALPHABET = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
 private static final int BASE = ALPHABET.length();

 private final AtomicLong sequenceGenerator;
 private final Map<String, UrlRecord> codeToUrlMap;
 private final Map<String, String> urlToCodeMap;

 public record UrlRecord(String originalUrl, String shortCode, Instant expirationTime) {}

 public TinyUrlEngine(long seedOffset) {
 this.sequenceGenerator = new AtomicLong(seedOffset);
 this.codeToUrlMap = new ConcurrentHashMap<>();
 this.urlToCodeMap = new ConcurrentHashMap<>();
 }

 public String shorten(String originalUrl, long ttlSeconds) {
```

```

 if (urlToCodeMap.containsKey(originalUrl)) {
 String existingCode = urlToCodeMap.get(originalUrl);
 UrlRecord record = codeToUrlMap.get(existingCode);
 if (record != null && record.expirationTime().isAfter(Instant.now())) {
 return existingCode;
 }
 }

 long uniqueId = sequenceGenerator.incrementAndGet();
 String code = encodeBase62(uniqueId);
 Instant expiration = (ttlSeconds > 0) ? Instant.now().plusSeconds(ttlSeconds) : J

 UrlRecord newRecord = new UrlRecord(originalUrl, code, expiration);
 codeToUrlMap.put(code, newRecord);
 urlToCodeMap.put(originalUrl, code);
 return code;
}

public String resolve(String shortCode) {
 UrlRecord record = codeToUrlMap.get(shortCode);
 if (record == null) {
 throw new IllegalArgumentException("Short code does not exist.");
 }
 if (Instant.now().isAfter(record.expirationTime())) {
 codeToUrlMap.remove(shortCode);
 urlToCodeMap.remove(record.originalUrl());
 throw new IllegalStateException("Short code expired.");
 }
 return record.originalUrl();
}

private String encodeBase62(long value) {
 StringBuilder sb = new StringBuilder();
 while (value > 0) {
 int remainder = (int) (value % BASE);

```

```

 sb.append(ALPHABET.charAt(remainder));
 value /= BASE;
 }
 return sb.reverse().toString();
}
}

```

## System 10: Thread-Safe In-Memory LRU Cache with TTL

### Design Decisions & Rationale

**Hash Map:** Delivers  $O(1)$  key-to-node lookups.

**Doubly-Linked List:** Enables  $O(1)$  node removals and head insertions.

**ReentrantReadWriteLock:** Grants concurrent read access while serializing write updates, preventing corruption under high concurrency.



### Complete Java Implementation

## Java



```
package com.system.lld.cache;

import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class ConcurrentLruCache<K, V> {

 private static class Node<K, V> {
 final K key;
 V value;
 long expireAt;
 Node<K, V> prev;
 Node<K, V> next;

 Node(K key, V value, long expireAt) {
 this.key = key;
 this.value = value;
 this.expireAt = expireAt;
 }

 boolean isExpired() {
 return expireAt > 0 && System.currentTimeMillis() > expireAt;
 }
 }

 private final int capacity;
 private final Map<K, Node<K, V>> index;
 private final Node<K, V> headSentinel;
 private final Node<K, V> tailSentinel;
 private final ReentrantReadWriteLock rwLock;
```

```
public ConcurrentLruCache(int capacity) {
 this.capacity = capacity;
 this.index = new HashMap<>(capacity);
 this.headSentinel = new Node<>(null, null, 0);
 this.tailSentinel = new Node<>(null, null, 0);
 headSentinel.next = tailSentinel;
 tailSentinel.prev = headSentinel;
 this.rwLock = new ReentrantReadWriteLock();
}

public V get(K key) {
 rwLock.writeLock().lock();
 try {
 Node<K, V> node = index.get(key);
 if (node == null) return null;

 if (node.isExpired()) {
 unlink(node);
 index.remove(key);
 return null;
 }

 moveToHead(node);
 return node.value;
 } finally {
 rwLock.writeLock().unlock();
 }
}

public void put(K key, V value, long ttlMs) {
 rwLock.writeLock().lock();
 try {
 long expireTime = (ttlMs > 0) ? System.currentTimeMillis() + ttlMs : 0;
 Node<K, V> existing = index.get(key);
```

```
 if (existing != null) {
 existing.value = value;
 existing.expireAt = expireTime;
 moveToHead(existing);
 } else {
 if (index.size() >= capacity) {
 Node<K, V> lru = tailSentinel.prev;
 unlink(lru);
 index.remove(lru.key);
 }
 Node<K, V> created = new Node<>(key, value, expireTime);
 attachHead(created);
 index.put(key, created);
 }
 } finally {
 rwLock.writeLock().unlock();
 }
}

private void attachHead(Node<K, V> node) {
 node.next = headSentinel.next;
 node.prev = headSentinel;
 headSentinel.next.prev = node;
 headSentinel.next = node;
}

private void unlink(Node<K, V> node) {
 node.prev.next = node.next;
 node.next.prev = node.prev;
}

private void moveToHead(Node<K, V> node) {
 unlink(node);
 attachHead(node);
}
```

```
}
}
```

## System 11: Multi-Elevator Dispatcher & Scheduler

### Design Decisions & Rationale

**Strategy Pattern:** Decouples elevator scheduling algorithms ( `LOOK` , `SCAN` , `Shortest-Seek` ) from operational state.

**LOOK Scheduling:** The car travels in its current direction, servicing all pending calls along the way, and reverses direction only when no further requests remain ahead.

### Complete Java Implementation

Java



```
package com.system.lld.elevator;

import java.util.*;

public class ElevatorController {

 public enum Direction { UP, DOWN, IDLE }

 public static class ElevatorCar {
 private final int id;
 private int currentFloor = 0;
 private Direction direction = Direction.IDLE;
 private final TreeSet<Integer> upRequests = new TreeSet<>();
 }
}
```

```
private final TreeSet<Integer> downRequests = new TreeSet<>(Collections.reverseOrder());

public ElevatorCar(int id) {
 this.id = id;
}

public synchronized void requestFloor(int destinationFloor) {
 if (destinationFloor == currentFloor) return;

 if (destinationFloor > currentFloor) {
 upRequests.add(destinationFloor);
 } else {
 downRequests.add(destinationFloor);
 }

 if (direction == Direction.IDLE) {
 direction = (destinationFloor > currentFloor) ? Direction.UP : Direction.DOWN;
 }
}

public synchronized void processNextStep() {
 if (direction == Direction.UP) {
 if (!upRequests.isEmpty()) {
 currentFloor = upRequests.pollFirst();
 } else if (!downRequests.isEmpty()) {
 direction = Direction.DOWN;
 currentFloor = downRequests.pollFirst();
 } else {
 direction = Direction.IDLE;
 }
 } else if (direction == Direction.DOWN) {
 if (!downRequests.isEmpty()) {
 currentFloor = downRequests.pollFirst();
 } else if (!upRequests.isEmpty()) {
 direction = Direction.UP;
 }
 }
}
```

```

 currentFloor = upRequests.pollFirst();
 } else {
 direction = Direction.IDLE;
 }
}

}

public int getCurrentFloor() { return currentFloor; }
public Direction getDirection() { return direction; }
public int getId() { return id; }
}

public interface DispatchStrategy {
 ElevatorCar findOptimalCar(List<ElevatorCar> cars, int sourceFloor, Direction dir)
}

public static class ClosestCarStrategy implements DispatchStrategy {
 @Override
 public ElevatorCar findOptimalCar(List<ElevatorCar> cars, int sourceFloor, Direction dir) {
 ElevatorCar bestCar = null;
 int lowestDelta = Integer.MAX_VALUE;

 for (ElevatorCar car : cars) {
 int delta = Math.abs(car.getCurrentFloor() - sourceFloor);
 if (car.getDirection() == Direction.IDLE || car.getDirection() == dir) {
 if (delta < lowestDelta) {
 lowestDelta = delta;
 bestCar = car;
 }
 }
 }
 return (bestCar != null) ? bestCar : cars.get(0);
 }
}

```

```
}
}
```

## System 12: Parking Lot Management System

### Design Decisions & Rationale

**Inheritance & Polymorphism:** Specializes parking spot dimensions ( `Compact` , `Large` , `Motorbike` ).

**Strategy Pattern:** Enables pluggable rate policies ( `HourlyRateStrategy` , `FlatRateStrategy` , `WeekendStrategy` ).

### Complete Java Implementation

Java



```
package com.system.lld.parkinglot;

import java.time.Duration;
import java.time.Instant;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class ParkingLotManager {

 public enum VehicleSize { MOTORBIKE, CAR, BUS }

 public abstract static class Vehicle {
```

```
private final String licensePlate;
private final VehicleSize size;

public Vehicle(String licensePlate, VehicleSize size) {
 this.licensePlate = licensePlate;
 this.size = size;
}

public VehicleSize getSize() { return size; }
public String getLicensePlate() { return licensePlate; }
}

public static class CompactCar extends Vehicle {
 public CompactCar(String plate) { super(plate, VehicleSize.CAR); }
}

public abstract static class ParkingSpot {
 private final String spotId;
 private boolean occupied;
 private Vehicle parkedVehicle;

 public ParkingSpot(String spotId) {
 this.spotId = spotId;
 }

 public abstract boolean fitsVehicle(Vehicle vehicle);

 public synchronized void park(Vehicle vehicle) {
 if (occupied || !fitsVehicle(vehicle)) {
 throw new IllegalStateException("Spot unavailable for vehicle sizing.");
 }
 this.parkedVehicle = vehicle;
 this.occupied = true;
 }
}
```

```

 public synchronized void vacate() {
 this.parkedVehicle = null;
 this.occupied = false;
 }

 public boolean isOccupied() { return occupied; }
 public String getSpotId() { return spotId; }
}

public static class CarSpot extends ParkingSpot {
 public CarSpot(String spotId) { super(spotId); }
 @Override
 public boolean fitsVehicle(Vehicle vehicle) {
 return vehicle.getSize() == VehicleSize.CAR || vehicle.getSize() == VehicleSi
 }
}

public record ParkingTicket(String ticketId, String spotId, String plate, Instant iss

public interface BillingStrategy {
 double computeFee(Duration elapsed);
}

public static class StandardHourlyBilling implements BillingStrategy {
 private final double ratePerHour;
 public StandardHourlyBilling(double ratePerHour) { this.ratePerHour = ratePerHour

 @Override
 public double computeFee(Duration elapsed) {
 long billableHours = Math.max(1, (long) Math.ceil(elapsed.toMinutes() / 60.0)
 return billableHours * ratePerHour;
 }
}

public static class ParkingLotFacility {

```

```

private final List<ParkingSpot> spots = new ArrayList<>();
private final Map<String, ParkingTicket> activeTickets = new ConcurrentHashMap<>()

public void registerSpot(ParkingSpot spot) { spots.add(spot); }

public synchronized ParkingTicket checkIn(Vehicle vehicle) {
 for (ParkingSpot spot : spots) {
 if (!spot.isOccupied() && spot.fitsVehicle(vehicle)) {
 spot.park(vehicle);
 String ticketId = UUID.randomUUID().toString();
 ParkingTicket ticket = new ParkingTicket(ticketId, spot.getSpotId(),
 activeTickets.put(ticketId, ticket));
 return ticket;
 }
 }
 throw new NoSuchElementException("Parking full for size: " + vehicle.getSize()
}

public synchronized double checkOut(String ticketId, BillingStrategy strategy) {
 ParkingTicket ticket = activeTickets.remove(ticketId);
 if (ticket == null) throw new IllegalArgumentException("Unknown ticket ID.");

 ParkingSpot assignedSpot = spots.stream()
 .filter(s -> s.getSpotId().equals(ticket.spotId()))
 .findFirst()
 .orElseThrow();

 assignedSpot.vacate();
 Duration elapsed = Duration.between(ticket.issuedAt(), Instant.now());
 return strategy.computeFee(elapsed);
}
}
}

```

# System 13: In-Memory Pub/Sub Engine

## Design Decisions & Rationale

**Observer Pattern:** Decouples message publishers from subscriber listener loops.

**Bounded Executor Pools:** Dispatches messages to subscribers asynchronously across dedicated thread pools, preventing slow consumers from blocking ingress pipelines.

## Complete Java Implementation

Java



```
package com.system.lld.pubsub;

import java.util.*;
import java.util.concurrent.*;

public class PubSubBus {

 public record EventMessage(String topic, String payload, long timestamp) {}

 public interface Subscriber {
 String getSubscriberId();
 void onEvent(EventMessage message);
 }

 public static class TopicChannel {
 private final String topicName;
 private final Set<Subscriber> subscribers = ConcurrentHashMap.newKeySet();
 private final ExecutorService dispatcher = Executors.newVirtualThreadPerTaskExecu
```

```

public TopicChannel(String topicName) {
 this.topicName = topicName;
}

public void subscribe(Subscriber subscriber) { subscribers.add(subscriber); }
public void unsubscribe(Subscriber subscriber) { subscribers.remove(subscriber); }

public void broadcast(EventMessage message) {
 for (Subscriber sub : subscribers) {
 dispatcher.submit(() -> {
 try {
 sub.onEvent(message);
 } catch (Exception ex) {
 System.err.printf("Error dispatching to %s: %s\n", sub.getSubscri
 }
 });
 }
}

private final Map<String, TopicChannel> channels = new ConcurrentHashMap<>();

public void createTopic(String topicName) {
 channels.putIfAbsent(topicName, new TopicChannel(topicName));
}

public void registerSubscriber(String topicName, Subscriber subscriber) {
 TopicChannel channel = channels.get(topicName);
 if (channel == null) throw new IllegalArgumentException("Topic does not exist: "
 channel.subscribe(subscriber);
}

public void emit(String topicName, String payload) {
 TopicChannel channel = channels.get(topicName);
 if (channel == null) throw new IllegalArgumentException("Topic does not exist: "

```

```

 channel.broadcast(new EventMessage(topicName, payload, System.currentTimeMillis())
 }
}

```

## System 14: Snake & Ladder Game Engine

### Design Decisions & Rationale

**Single Responsibility Principle (SRP):** Separates board coordinates and transition links ( Snake , Ladder ) from game loop orchestration and dice behaviors.

**Deterministic Testability:** The DiceRollStrategy interface supports pluggable random-source implementations, simplifying automated unit testing.

### Complete Java Implementation

Java



```

package com.system.lld.snakeladder;

import java.util.*;

public class SnakeAndLadderGame {

 public record BoardJump(int fromCell, int toCell) {
 public BoardJump {
 if (fromCell == toCell) throw new IllegalArgumentException("Start cannot match End")
 }
 }
}

```

```
public static class GameBoard {
 private final int totalCells;
 private final Map<Integer, Integer> jumpMap = new HashMap<>();

 public GameBoard(int totalCells) {
 this.totalCells = totalCells;
 }

 public void setSnake(int head, int tail) {
 if (head <= tail) throw new IllegalArgumentException("Snake head must be above tail");
 jumpMap.put(head, tail);
 }

 public void setLadder(int start, int end) {
 if (start >= end) throw new IllegalArgumentException("Ladder base must be below top");
 jumpMap.put(start, end);
 }

 public int calculateNextPosition(int targetPosition) {
 return jumpMap.getOrDefault(targetPosition, targetPosition);
 }

 public int getTotalCells() { return totalCells; }
}

public static class PlayerToken {
 private final String name;
 private int currentSquare = 0;

 public PlayerToken(String name) { this.name = name; }
 public String getName() { return name; }
 public int getCurrentSquare() { return currentSquare; }
 public void setSquare(int s) { this.currentSquare = s; }
}
```

```
public interface DiceStrategy {
 int roll();
}

public static class StandardDie implements DiceStrategy {
 private final Random random = new Random();
 @Override
 public int roll() { return random.nextInt(6) + 1; }
}

public static class GameMatch {
 private final GameBoard board;
 private final DiceStrategy dice;
 private final Deque<PlayerToken> players = new ArrayDeque<>();
 private boolean winnerAnnounced = false;

 public GameMatch(GameBoard board, DiceStrategy dice, List<PlayerToken> participar
 this.board = board;
 this.dice = dice;
 this.players.addAll(participants);
 }

 public boolean advanceTurn() {
 if (winnerAnnounced) return true;

 PlayerToken current = players.pollFirst();
 int roll = dice.roll();
 int dest = current.getCurrentSquare() + roll;

 if (dest <= board.getTotalCells()) {
 dest = board.calculateNextPosition(dest);
 current.setSquare(dest);
 }
 }
}
```

```
 if (current.getCurrentSquare() == board.getTotalCells()) {
 System.out.println("Player has won the match: " + current.getName());
 winnerAnnounced = true;
 return true;
 }

 players.addLast(current);
 return false;
 }
}
```

## System 15: Concurrency-Safe Movie Ticket Reservation System (BookMyShow)

### Design Decisions & Rationale

**Two-Phase Reservation Lifecycle:** Direct commits during seat selection cause booking collisions.

*Phase 1 (Temporary Lock):* Selected seats transition to **LOCKED** with an expiration window (e.g., 600 seconds) using thread-safe CAS operations.

*Phase 2 (Payment Confirmation):* Confirmed purchases mark seats **BOOKED** . If payments fail or sessions time out, locks clear and seats return to **AVAILABLE** .

### Complete Java Implementation

Java



```
package com.system.lld.booking;

import java.time.Instant;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class TicketReservationManager {

 public enum SeatState { AVAILABLE, LOCKED, BOOKED }

 public static class SeatEntity {
 private final String seatId;
 private SeatState state = SeatState.AVAILABLE;
 private Instant lockReleaseDeadline = Instant.MIN;

 public SeatEntity(String seatId) { this.seatId = seatId; }

 public synchronized boolean reserveLock(long timeoutSeconds) {
 if (state == SeatState.BOOKED) return false;

 if (state == SeatState.LOCKED && Instant.now().isBefore(lockReleaseDeadline))
 return false; // Still actively locked by an in-flight checkout
 }

 this.state = SeatState.LOCKED;
 this.lockReleaseDeadline = Instant.now().plusSeconds(timeoutSeconds);
 return true;
 }

 public synchronized boolean commitBooking() {
 if (state == SeatState.LOCKED && Instant.now().isBefore(lockReleaseDeadline))
 this.state = SeatState.BOOKED;
 return true;
 }
 }
}
```

```
 return false;
 }

 public synchronized void unlock() {
 if (state != SeatState.BOOKED) {
 this.state = SeatState.AVAILABLE;
 this.lockReleaseDeadline = Instant.MIN;
 }
 }

 public SeatState getState() { return state; }
 public String getSeatId() { return seatId; }
}

public static class BookingShowCoordinator {
 private final Map<String, SeatEntity> seatInventory = new ConcurrentHashMap<>();

 public void provisionSeat(String seatId) {
 seatInventory.put(seatId, new SeatEntity(seatId));
 }

 public boolean lockSeatSelection(List<String> seatIds, long timeoutSeconds) {
 List<SeatEntity> lockedProgress = new ArrayList<>();
 for (String sid : seatIds) {
 SeatEntity seat = seatInventory.get(sid);
 if (seat != null && seat.reserveLock(timeoutSeconds)) {
 lockedProgress.add(seat);
 } else {
 // Roll back previously acquired locks if any seat fails
 lockedProgress.forEach(SeatEntity::unlock);
 return false;
 }
 }
 return true;
 }
}
```

```
public boolean confirmTransaction(List<String> seatIds) {
 for (String sid : seatIds) {
 SeatEntity seat = seatInventory.get(sid);
 if (seat == null || !seat.commitBooking()) {
 return false;
 }
 }
 return true;
}
```

# Module 5: Comparative Trade-Off Matrices & Architectural Synthesis

| SYSTEM TRADE-OFF SUMMARY |                    |                    |                           |
|--------------------------|--------------------|--------------------|---------------------------|
| Problem Domain           | Primary Bottleneck | Primary Datastore  | Concurrency / Consistency |
| Media Streaming          | Outbound Egress    | S3 + Global CDNs   | Event-driven transcode    |
| Geo-Dispatch             | Ingest Write Rate  | Redis + H3 Index   | Spatial sharding by c     |
| Order Systems            | Data Contention    | MySQL + PostgreSQL | Redis token pre-alloc     |
| Instant Messaging        | Open TCP Streams   | ScyllaDB (Inboxes) | Ephemeral socket conr     |
| Social Feeds             | Celebrity Fanout   | Redis + TiDB       | Hybrid Push-Pull pipe     |

| Rate Limiting | Low-Latency Access | In-Memory Redis | Atomic Lua script eval  
+-----+-----+-----+-----+

## Module 6: Self-Study Platform Web Architecture

[ Next.js 14 Frontend Application (TailwindCSS, Markdown Renderer) ]

|  
(HTTPS REST / SSE Event Streams)

v

[ FastAPI Microservice Cluster (Python 3.11 / Uvicorn) ]

|  
(PostgreSQL 16)

JSONB Curricula

|  
(ChromaDB / pgvector)

Vector Documentation

|  
(Redis 7)

Session & Cache

**Frontend:** Next.js application delivering static notes, system design blueprints, and interactive code walkthroughs.

**Backend:** FastAPI microservice managing system design notes, a RAG-based search engine, and an integrated RAG-based search engine. i wanted a pdf for all this

**Data Layer:** PostgreSQL (JSONB) for modular curricula data, Redis for cached architectural templates, and a vector datastore for intelligent document retrieval.

The comprehensive study notes covering the full lecture series and fundamentals are compiled into the reference manual:

### 7. How to Export This Notebook as a PDF

**File Name:** deep-learning-masterclass-notes-v2.pdf

To save MD ; entire guide as a clean, publication-ready PDF:

Location: Saved directly inside your Studio panel as a permanent artifact. MD

|                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What is Included Inside This PDF Reference Manual: |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 1.                                                 | <div><div>Title &amp; Theoretical Foundations:</div><div>Option 1: Browser Print Preview</div><div><div>1. Open this view in your browser.</div><div>2. Press Ctrl + P (Windows/Linux) or Cmd + P (macOS).</div><div>3. Destination: "Save as PDF".</div><div>4. Margins: "Default" or "Minimum"; check "Background graphics".</div></div><div>Option 2: CLI Export via randoc / Weasyprint</div><div>2.</div><div><div>Perceptron, Mathematics &amp; Propagation.</div><div>Single-layer Perceptrons and the forward propagation sequence <math>z = \mathbf{w}^T \mathbf{x} + b</math>, followed by activation <math>a = \sigma(z)</math>.</div><div>Why biases are necessary (preventing null outputs when weights initialize to zero).</div><div>YouTube</div><div>The chain rule of differentiation for backpropagation across multi-path and multi-layer architectures.</div><div>YouTube</div></div><div>3.</div><div><div>Loss Functions &amp; Vanishing Gradient Taxonomy:</div><div>Mathematical derivations and formulas for Regression Losses (MSE, MAE, Huber) and Classification Losses (Binary Cross-Entropy / Log Loss, Categorical Cross-Entropy, Softmax).</div><div>Detailed breakdown of the vanishing gradient problem in deep networks using Sigmoid/Tanh.</div><div>Full taxonomy of modern activation functions (ReLU, Leaky ReLU, PReLU, ELU, Swish).</div><div>YouTube</div></div><div>4.</div><div><div>Complete Optimizer Suite:</div></div></div> |

Comparative mathematical derivations: Batch Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch SGD, SGD with Momentum (exponential moving averages), AdaGrad, RMSProp, and Adam. [MD +1](#)

## 5. Code Implementations & Computer Vision (CNNs):

**ANN Implementation:** Complete TensorFlow/Keras pipeline for tabular churn classification, featuring feature scaling, Dropout layers, and Early Stopping callbacks

. [YouTube](#)

**CNN Architectures:** Mathematics of 2D convolutions, spatial dimension formulas  $\left( \frac{n+2p-f}{s} + 1 \right)$ , padding, stride mechanics, Max Pooling vs. Average Pooling, flattening, and a full CIFAR-10 classification script. [YouTube](#)

You can open and download [deep-learning-masterclass-notes-v2.pdf](#) directly from the Studio panel on your screen. [MD](#)